

プラットフォーム向け送り状発行API

API仕様書

Ver. 1.10

佐川急便株式会社

改版履歴

版数	改版日付	改版内容
1.00	2022/8/1	初版
1.10	2025/7/28	即時発行API追加

CONTENTS

改版履歴

1. はじめに

2. 共通定義

3. API一覧

4. インターフェース定義

5. コード定義

1.はじめに

1.1. 本書について

本書「API仕様書」は、佐川急便株式会社(以下、「佐川急便」)が提供するAPIの仕様内容、I/Fを記述したものです。

APIは各機能ごとにURLを分けており、利用したい機能のURLに対してBODY部に認証情報とパラメータをXMLまたはJSON形式で送信することで処理が行われます。

1.2. 概要

本APIは送り状発行を支援するサービスとなります。

出荷情報をパラメータとして送信することで送り状のPDFを取得することができます。

住所チェックなど送り状出力の可否チェックも行うため送り状帳票作成の負担を軽減することができます。

1.3. 注意事項

APIで提供する帳票は佐川急便の仕様に基づいております。

佐川急便の変更に合わせて帳票の仕様を変更することがあります。

代引契約、運送保険のご利用は別途配送に関するご契約が必要となります。

1.はじめに

1.4. 帳票レイアウト

APIで提供している送り状は以下の種類となります。

帳票レイアウトによって指定された文字、項目が全て印字されない場合があります。

詳細は各帳票レイアウトをご確認ください。

■ 佐川急便A5サイズ圧着式送り状

配達に関する各種ケアマークを送り状に印字することができるため、
ご指定事項の多いお客様に最適です。



■ 佐川急便統一圧着サーマル送り状

最も標準的な送り状です。
専用プリンタをご利用ください。



■ 佐川急便ケアマーク入圧着式送り状

ケアマークがプレ印字された送り状のため、シールを貼る手間が省けます。
専用プリンタをご利用ください。



1.はじめに

1.4.1. 透かし文字の有無

APIで提供している送り状種類毎に

「透かし文字有」版、「透かし文字無」版レイアウトのご利用が可能です。

利用レイアウトはリクエストされた送り状コードより判定されます。

送り状コードについては 4.インターフェース定義、5.1.パラメーターコード定義をご確認ください。

透かし文字：Acrobat Reader上でPDFプレビュー表示時に下図イメージ赤枠部分として表示されている、

佐川急便専用用紙を用いた印刷が必要な旨を示す文言です（印刷時に印字はされません）

（参考イメージ_佐川急便A5サイズ圧着式送り状）

■ PDFプレビュー「透かし文字有」版



■ 実印刷イメージ「透かし文字有」版



■ PDFプレビュー「透かし文字無」版



■ 実印刷イメージ「透かし文字無」版



2.共通定義

2.1. ログイン認証

各APIを利用する際はその都度、ログイン認証を行います。

認証は下記の項目を用います。

項番	項目名	タグ名	型	長さ	備考
1	カスタマーID	customerId	半角文字	50	佐川急便で発行する顧客のID
2	ログインパスワード	loginPassword	半角文字	100	佐川急便で発行する顧客のパスワード

※ カスタマーID/ログインパスワードは佐川急便が発行致します。

2.1.1. カスタマーID

本システムを利用するお客様単位に発行します。

カスタマーIDの編集はできません。

2.1.2. ログインパスワード

カスタマーIDの情報を暗号化した文字列をパスワードとします。

パスワードは編集できません。忘れた際には再発行の手続きが必要です。

2.共通定義

2.2. インターフェース一覧項目表

必須項目 ◎ ： 必須項目
 ○ ： 準必須項目（条件によっては必須となる項目）

項目カラー

赤：	◎となっている必須項目
黄：	○となっている準必須項目
青：	親ノード ※下位層項目を設定する場合は必須項目

属性

文字の形式を各項目で指定しています。属性の内容に従って値をセットして下さい。

項番	属性	内容
1	半角数字	[0-9]までの数字
2	半角文字	ASCIIの10進数32~126まで [0-9,a-z,A-Z,@-/…etc.]
3	指定なし	UTF-8でサポートする文字全般

2.3. PDF動作環境

本APIから発行されるPDFの表示・印字には、Adobe Acrobat Reader(最新版)のご利用を推奨しております。
Adobe Acrobat Reader以外のプラグインでは、正しく表示・印字できない場合があります。

3.API一覧

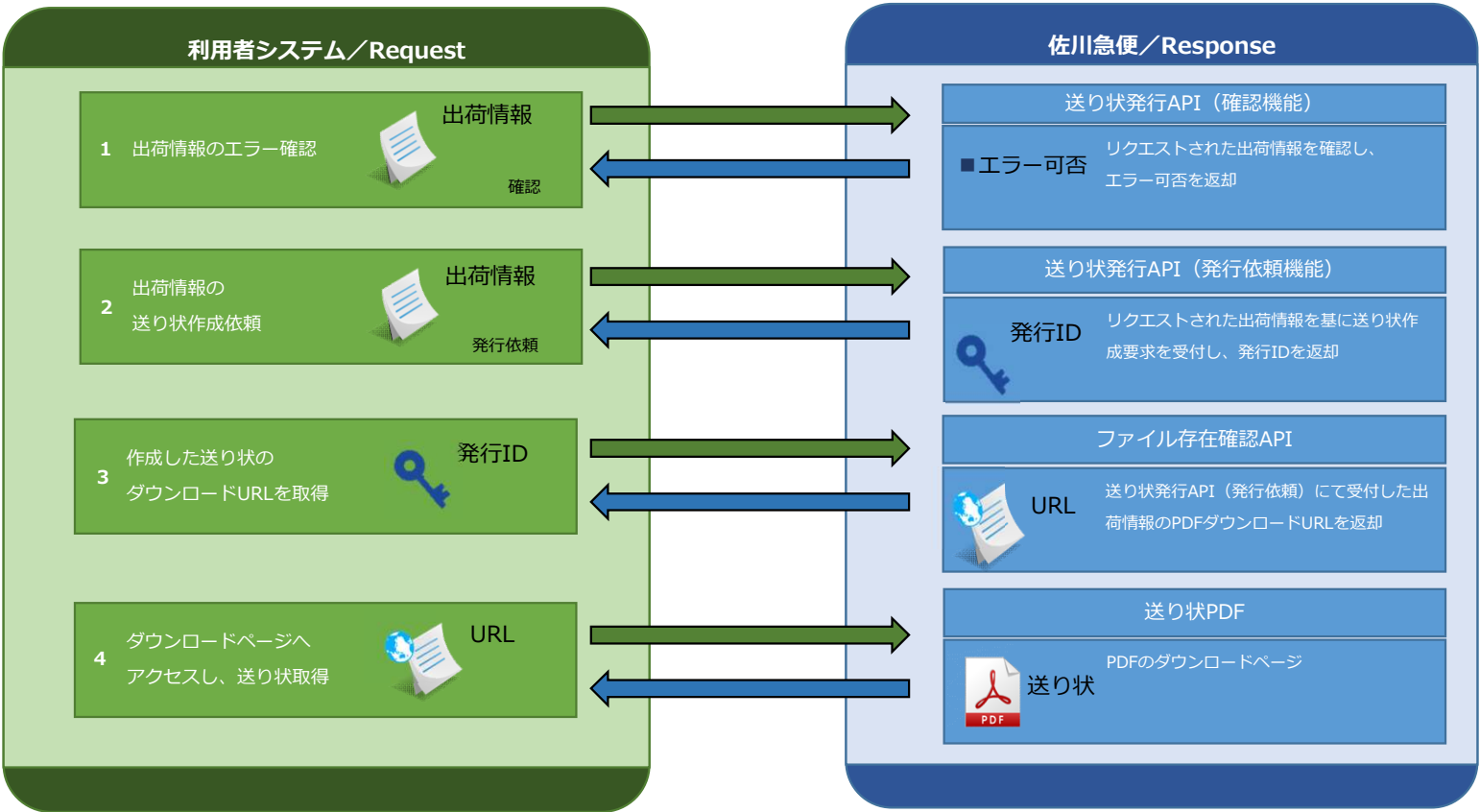
3.1. API一覧

項番	API名	機能	概要
1	送り状発行API	確認	送り状を出力する条件を満たしているかチェック処理を行う
		発行依頼	送り状発行API（確認）同様にチェック処理を行い、条件を満たした場合PDF作成要求を行う
2	即時発行API	確認	送り状を出力する条件を満たしているかチェック処理を行う
		発行依頼	即時発行API（確認）同様にチェック処理を行い、条件を満たした場合PDF作成要求・URL返却を行う
3	ファイル存在確認API	確認/取得	送り状発行API（発行依頼）にてPDFデータが作成されたかどうか、ファイルチェックを行う
4	利用実績API	確認	PDF作成した実績を取得する
5	送り状再発行依頼API	発行	過去にPDF作成完了した情報を再度PDF作成要求を行う
6	佐川急便マスタ参照API	取得	佐川急便の営業店コードの取得やリクエストした郵便番号と住所の判定を行う
7	荷物受渡書・出荷明細書発行API	発行依頼	荷物受渡書・出荷明細書PDFの作成要求を行う

3.API一覧

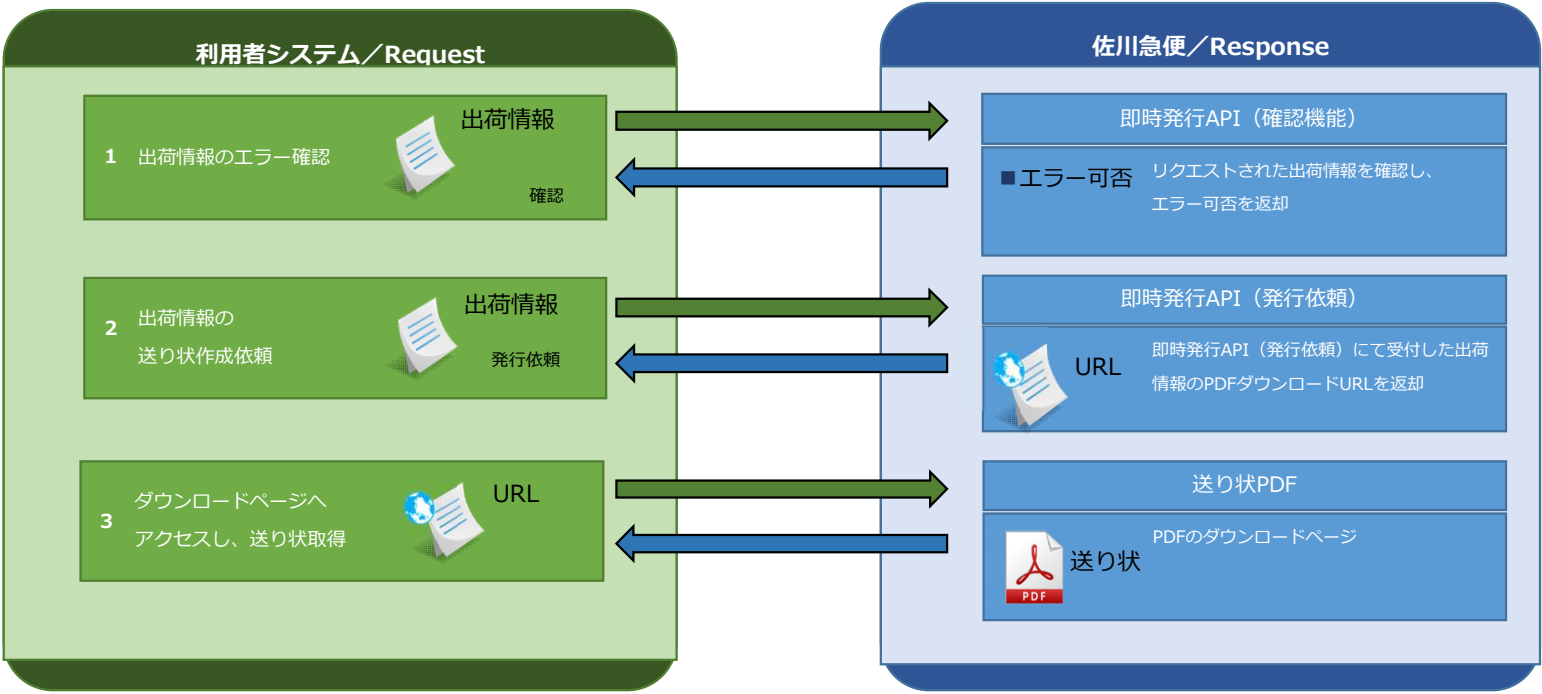
3.2. API利用イメージ

3.2.1. (送り状発行API利用の場合)送り状PDF出力までのフロー



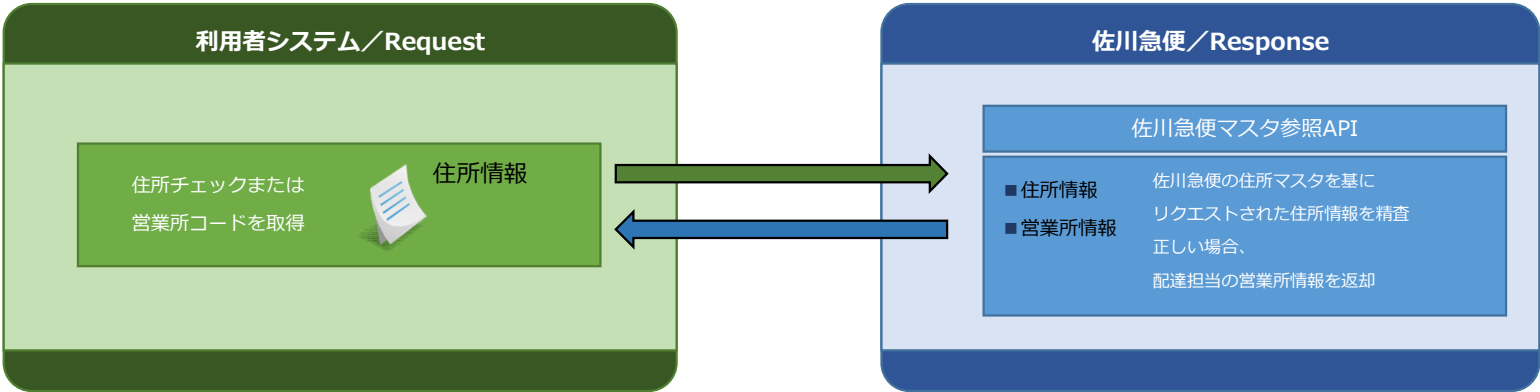
3.API一覧

3.2.1. (即時発行API利用の場合)送り状PDF出力までのフロー

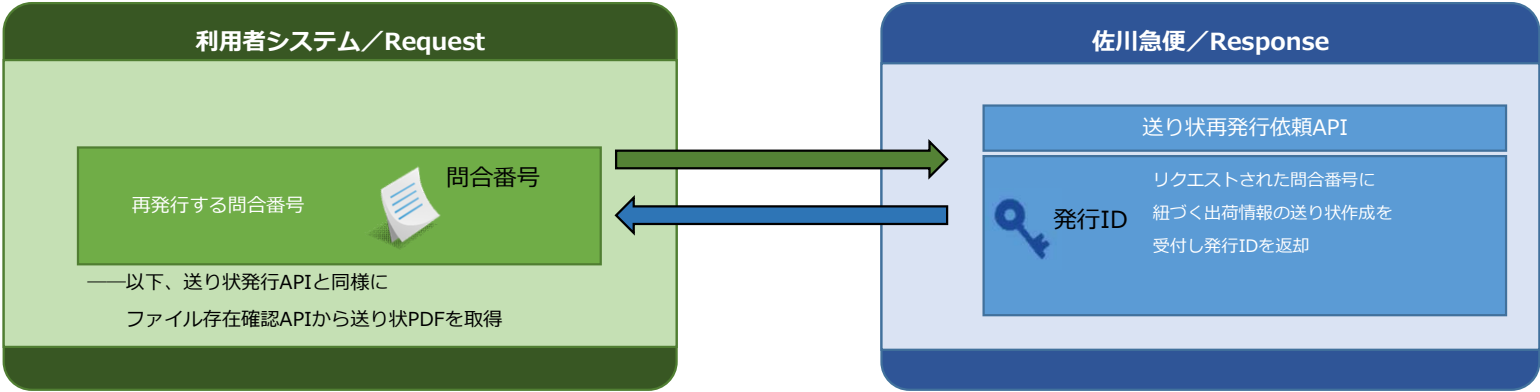


3.API一覧

3.2.2. 郵便番号、住所の不一致確認および配達担当の営業所情報を取得

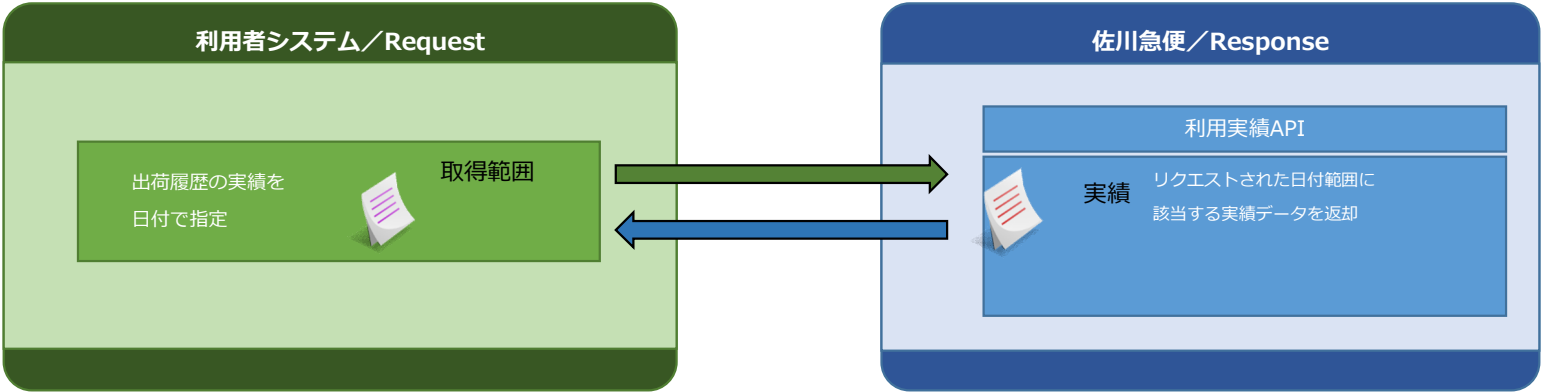


3.2.3. 送り状の再発行

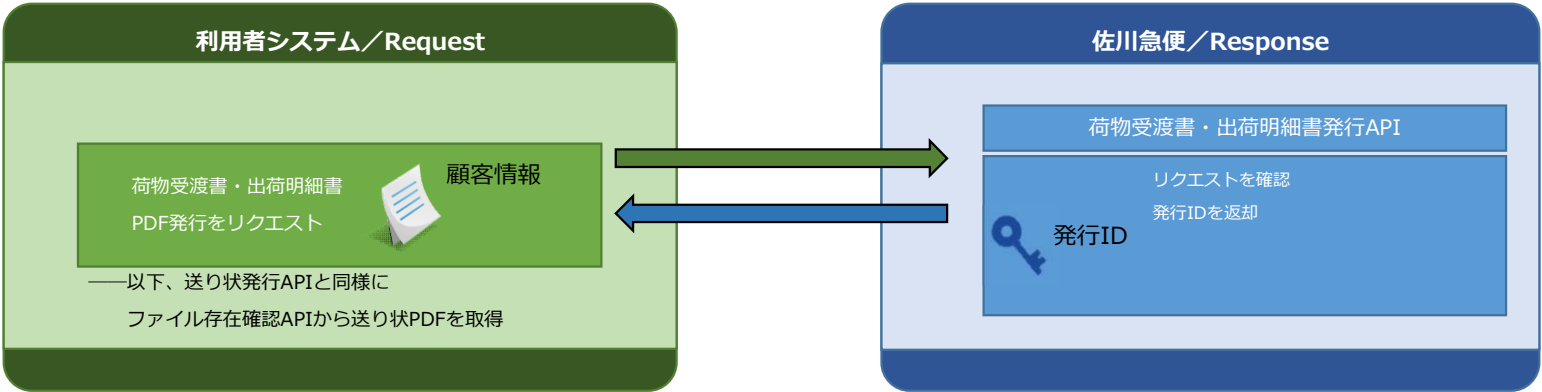


3.API一覧

3.2.4. 利用実績を取得



3.2.5. 荷物受渡書・出荷明細書の発行



3.API一覧

3.3. 共通定義

各APIの共通項目となります。

項目	内容
プロトコル	HTTPS(TLS1.2/TLS1.3)
文字コード	UTF-8
Content-Type	application/xml, application/json

3.4. 各API

3.4.1. 送り状発行API(確認機能)

「送り状発行（確認機能）API」では、送り状出力対象の出荷情報に対し、出力条件を満たしているか確認処理を行います。

以下のチェック項目を確認します。

チェック項目	内容
住所チェック	郵便番号および住所が一致していることを確認 ※佐川急便の郵便番号マスタを用いて住所チェックを行う
地域チェック	配達指定日、時間帯希望、クール便など対応可能地域であることを確認 配達する際、中継があることを確認
文字列チェック	送り状印字の規定文字数であることを確認

チェック結果はコードで返却します。

リクエスト形式	RequestURL
XML	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/shipping/xml
JSON	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/shipping/json
項目	内容
HTTPメソッド	POST
リクエスト	精査対象の出荷情報(一度に複数出荷情報のリクエストが可能)
レスポンス	出荷情報単位で送り状出力条件確認した処理結果コードを返却
インターフェース	4.1.送り状発行API I/F参照

3.API一覧

3.4.2. 送り状発行API(発行依頼機能)

「送り状発行API（発行依頼機能）」では、リクエストで受けた出荷情報を基に送り状のPDFの作成要求を行います。
「送り状発行API（確認機能）」と同様にチェック処理を行い、送り状発行条件を満たした場合、発行受付IDを返却します。

リクエスト形式	RequestURL
XML	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/shipping/xml
JSON	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/shipping/json
項目	内容
HTTPメソッド	POST
リクエスト	送り状出力対象の出荷情報(一度に複数出荷情報のリクエストが可能)
レスポンス	全リクエストデータが送り状発行条件を満たす場合、問合番号、発行受付IDを返却 ※複数出荷情報リクエストの内、1件でもエラーが存在する場合は発行受付IDは返却しない エラーが存在する場合は「送り状発行API（確認機能）」と同様に処理結果コードを返却
インターフェース	4.1.送り状発行API I/F参照

3.API一覧

3.4.3. 即時発行API(確認機能)

「送り状発行（確認機能）API」では、送り状出力対象の出荷情報に対し、出力条件を満たしているか確認処理を行います。
以下のチェック項目を確認します。

チェック項目	内容
住所チェック	郵便番号および住所が一致していることを確認 ※佐川急便の郵便番号マスタを用いて住所チェックを行う
地域チェック	配達指定日、時間帯希望、クール便など対応可能地域であることを確認 配達する際、中継があることを確認
文字列チェック	送り状印字の規定文字数であることを確認

チェック結果はコードで返却します。

リクエスト形式	RequestURL
XML	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/sokuji/xml
JSON	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/sokuji/json

項目	内容
HTTPメソッド	POST
リクエスト	精査対象の出荷情報(一度に複数出荷情報のリクエストが可能)
レスポンス	出荷情報単位で送り状出力条件確認した処理結果コードを返却
インターフェース	4.1.即時発行API I/F参照

3.API一覧

3.4.4. 即時発行API(発行依頼機能)

「即時発行API（発行依頼機能）」では、リクエストで受けた出荷情報を基に送り状のPDFの作成要求を行います。

「即時発行API（確認機能）」と同様にチェック処理を行い、送り状発行条件を満たした場合、ダウンロード先のURLを返却します。

リクエスト形式	RequestURL
XML	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/sokuji/xml
JSON	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/sokuji/json
項目	内容
HTTPメソッド	POST
リクエスト	送り状出力対象の出荷情報(一度に複数出荷情報のリクエストが可能)
レスポンス	全リクエストデータが送り状発行条件を満たす場合、問合番号、PDFダウンロードURLを返却 ※複数出荷情報リクエストの内、1件でもエラーが存在する場合は送り状URLは返却しない エラーが存在する場合は「即時発行API（確認機能）」と同様に処理結果コードを返却
インターフェース	4.2.即時発行API I/F参照

3.API一覧

3.4.5. ファイル存在確認API

「ファイル存在確認API」では「送り状発行API（発行依頼機能）」要求に対し、送り状PDF発行結果を返却します。

送り状PDF作成完了後、ダウンロード先のURLを返却します。

URLの取得は生成後、24時間取得可能です。24時間を超えた場合は再生成が必要となります。

リクエスト形式	RequestURL
XML	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/checkfile/xml
JSON	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/checkfile/json
項目	内容
HTTPメソッド	POST
リクエスト	送り状発行API（発行依頼機能）で取得した発行受付ID
レスポンス	発行受付IDに紐づくPDFダウンロードURLを返却 PDF作成未完了の場合は対象の処理結果コードを返却
インターフェース	4.2.ファイル存在確認API I/F参照

3.API一覧

3.4.6. 利用実績API

「利用実績API」では指定の日付範囲からPDF作成した実績情報を返却します。

API使用日より365日前までの日付範囲の指定が可能です。

リクエスト形式	RequestURL
XML	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/riyoujisseki/xml
JSON	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/riyoujisseki/json
項目	内容
HTTPメソッド	POST
リクエスト	取得対象の日付範囲
レスポンス	PDF発行した件数を返却
インターフェース	4.3.利用実績参照API I/F参照

3.4.7. 送り状再発行依頼API

「送り状発行API（発行依頼機能）」にて過去に作成完了した出荷情報を、再度送り状PDF作成を行う場合に使用します。

「送り状再発行依頼API」では送り状発行完了した出荷情報を、同じ問合番号で再度作成します。

再発行依頼は初回発行要求から72時間まで有効です。

※出荷情報の訂正の場合は「送り状発行API（発行依頼機能）」を使用し、新規に問合番号を取得して下さい。

「送り状再発行依頼API」にて発行した送り状には、再発行のマークが表示されます。

リクエスト形式	RequestURL
XML	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/retryprint/xml
JSON	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/retryprint/json
項目	内容
HTTPメソッド	POST
リクエスト	再発行する出荷情報の問合番号
レスポンス	「送り状発行API（発行依頼機能）」同様 初回「送り状発行API（発行依頼機能）」要求から72時間を経過している問合番号は対象外
インターフェース	4.4.送り状再発行依頼API I/F参照

3.API一覧

3.4.8. 佐川急便マスタ参照API

佐川急便の郵便番号マスタを基に住所精査を行います。

リクエストした郵便番号、住所情報の確認及び、住所情報の配送担当地区となる営業所情報を返却します。

リクエスト形式	RequestURL
XML	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/checkaddress/xml
JSON	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/checkaddress/json
項目	内容
HTTPメソッド	POST
リクエスト	郵便番号、住所情報
レスポンス	リクエストされた郵便番号と住所情報の整合性確認し結果返却 一致の場合、担当の営業所情報を返却 不一致の場合は、郵便番号に該当する住所情報、住所情報に該当する郵便番号を返却
インターフェース	4.5.佐川急便マスタ参照API I/F参照

3.4.9. 荷物受渡書・出荷明細書発行API

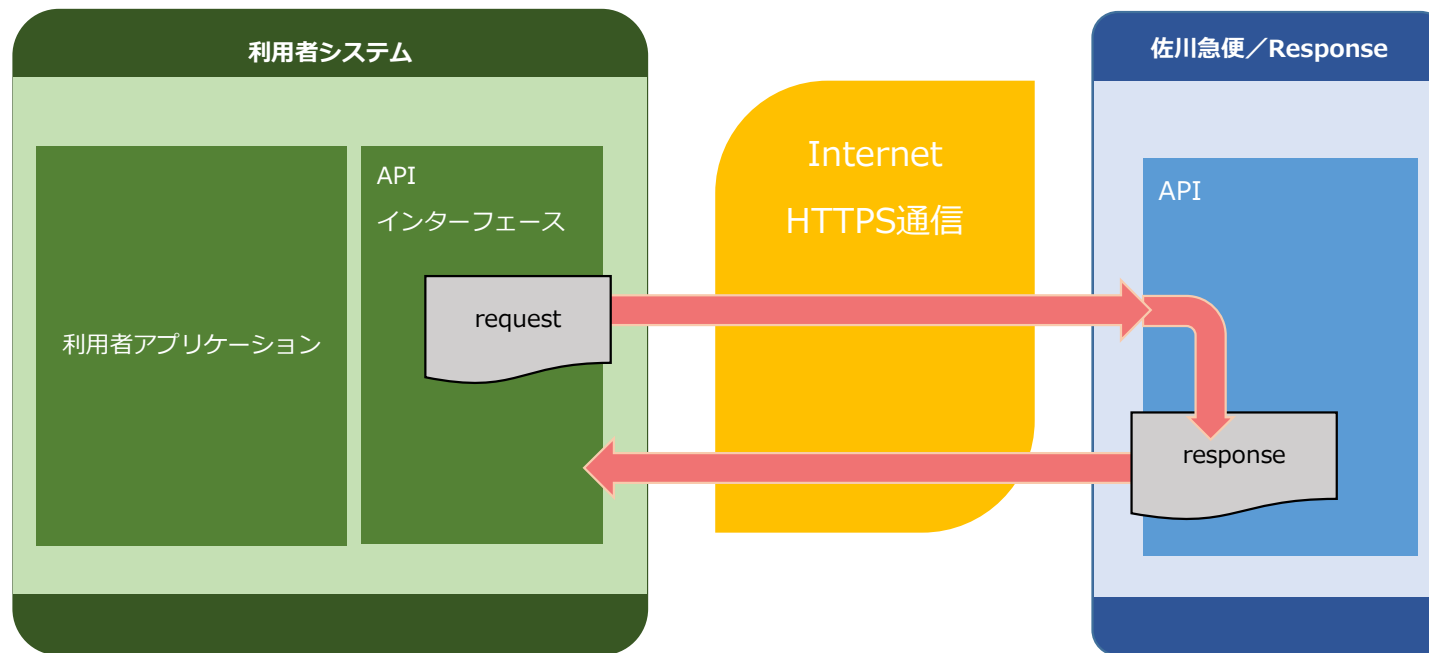
「荷物受渡書・出荷明細書発行API」では、荷物出荷時にご利用頂く荷物受渡書と、出荷明細書の発行を行います。

出荷明細情報の取得期限は、送り状発行要求した日時から90日後までとなります。

リクエスト形式	RequestURL
XML	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/ukewatashimeisai/xml
JSON	https://dummy.sagawa-exp.co.jp/rest/insatsumodule/ukewatashimeisai/json
項目	内容
HTTPメソッド	POST
リクエスト	顧客コード、荷物出荷情報
レスポンス	発行受付IDを返却
インターフェース	4.6.荷物受渡書・出荷明細書発行API I/F参照

3.API一覧

3.5. API利用方法



3.API一覧

3.5.1 JavaScriptを用いたAPI利用方法

3.5.1.1. XML形式でリクエストした場合

```
// XMLドキュメントをサーバーからロード、APIへXMLドキュメントを渡す方法

// XMLHttpRequestのオブジェクトを用意
var ajax = !window.XMLHttpRequest && window.ActiveXObject ? new ActiveXObject('Microsoft.XMLHTTP') : new XMLHttpRequest();
var urlApiShipping = 'https://dummy.sagawa-exp.co.jp/rest/insatsumodule/shipping/xml'; // 送り状発行APIのURL

var xmlDocRequest = ...; // リクエストXMLを用意する

ajax.open('POST', urlApiShipping, /*async*/false);
ajax.setRequestHeader('Content-Type', 'application/xml');
ajax.send(xmlDocRequest); // ダウンロードしたXMLをAPIへ送る
if (ajax.status != 200) {
    return false; // エラーがある
}

var xmlDocResponse = ajax.responseXML; // APIからのレスポンスXMLを貰う
... // 取得したレスポンスのXMLのチェック等の処理を行う
```

3.API一覧

3.5.1.2. JSON形式でリクエストした場合

// JSONドキュメントをサーバーからロード、APIへJSONドキュメントを渡す方法

// JSONHttpRequestのオブジェクトを用意

var ajax = !window.JSONHttpRequest && window.ActiveXObject ? new ActiveXObject('Microsoft.JSONHTTP') : new JSONHttpRequest();

var urlApiShipping = 'https://dummy.sagawa-exp.co.jp/rest/insatsumodule/shipping/json'; // 送り状発行APIのURL

var jsonDocRequest = ...; // リクエストJSONを用意する

ajax.open('POST', urlApiShipping, /*async*/false);

ajax.setRequestHeader('Content-Type', 'application/json');

ajax.send(jsonDocRequest); // ダウンロードしたJSONをAPIへ送る

if (ajax.status != 200) {

return false; // エラーがある

}

var jsonDocResponse = ajax.responseJSON; // APIからのレスポンスJSONを貰う

... // 取得したレスポンスのJSONのチェック等の処理を行う

3.API一覧

3.5.1.3. XML形式でリクエストした場合(即時発行APIを利用する場合のサンプル)

```
// XMLドキュメントをサーバーからロード、APIへXMLドキュメントを渡す方法

// XMLHttpRequestのオブジェクトを用意
var ajax = !window.XMLHttpRequest && window.ActiveXObject ? new ActiveXObject('Microsoft.XMLHTTP') : new XMLHttpRequest();
var urlApiShipping = 'https://dummy.sagawa-exp.co.jp/rest/insatsumodule/sokuji/xml'; // 即時発行APIのURL

var xmlDocRequest = ...; // リクエストXMLを用意する

ajax.open('POST', urlApiShipping, /*async*/false);
ajax.setRequestHeader('Content-Type', 'application/xml');
ajax.send(xmlDocRequest); // ダウンロードしたXMLをAPIへ送る
if (ajax.status != 200) {
  return false; // エラーがある
}

var xmlDocResponse = ajax.responseXML; // APIからのレスポンスXMLを貰う
... // 取得したレスポンスのXMLのチェック等の処理を行う

var responseUrl = xmlDocResponse.getElementsByTagName("url"); //レスポンスXMLからurlタグ要素を取得
async function fetchWithRetry(url, options = {}, retries = ..., delay = ...) { //取得したurlを確認、リトライ処理を追加
  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      let response = await fetch(url, options);
      if (response.ok) {
        return response; // ステータスコードが成功範囲内の場合
      } else {
        ... //エラー処理を行う
      }
    } catch {
      await new Promise((resolve) => setTimeout(resolve, delay));
    }
  }
  ... //リトライ失敗時のエラー処理
}
```

3.API一覧

3.5.1.4. JSON形式でリクエストした場合(即時発行APIを利用する場合のサンプル)

```
// JSONドキュメントをサーバーからロード、APIへJSONドキュメントを渡す方法

// JSONHttpRequestのオブジェクトを用意
var ajax = !window.JSONHttpRequest && window.ActiveXObject ? new ActiveXObject('Microsoft.JSONHTTP') : new JSONHttpRequest();
var urlApiShipping = 'https://dummy.sagawa-exp.co.jp/rest/insatsumodule/sokuji/json'; // 即時発行APIのURL

var jsonDocRequest = ...; // リクエストJSONを用意する

ajax.open('POST', urlApiShipping, /*async*/false);
ajax.setRequestHeader('Content-Type', 'application/json');
ajax.send(jsonDocRequest); // ダウンロードしたJSONをAPIへ送る
if (ajax.status != 200) {
    return false; // エラーがある
}

var jsonDocResponse = ajax.responseJSON; // APIからのレスポンスJSONを貰う
... // 取得したレスポンスのJSONのチェック等の処理を行う

var responseUrl = responseJSON.printDataList.printDataDetail[0].url; //レスポンスJSONからurl要素を取得
async function fetchWithRetry(url, options = {}, retries = ..., delay = ...) { //取得したurlを確認、リトライ処理を追加
    for (let attempt = 1; attempt <= retries; attempt++) {
        try {
            let response = await fetch(url, options);
            if (response.ok) {
                return response; // ステータスコードが成功範囲内の場合
            } else {
                ... //エラー処理を行う
            }
        } catch {
            await new Promise((resolve) => setTimeout(resolve, delay));
        }
        ... //リトライ失敗時のエラー処理
    }
}
```

3.API一覧

3.5.2 Javaを用いたAPI利用方法

3.5.2.1. XML形式でリクエストした場合

```
// APIへXMLドキュメントを渡す方法
URL urlSend = new URL("https://dummy.sagawa-exp.co.jp/rest/insatsumodule/shipping/xml"); // 送り状発行APIのURL
URLConnection connSend = (URLConnection)urlSend.openConnection();
connSend.setDoOutput(true); // APIへ送るデータ
connSend.setDoInput(true); // APIから貰うデータ
connSend.setRequestMethod("POST"); // POSTが必須
connSend.setRequestProperty("Content-Type","application/xml"); // 送るデータの形式はXML

connSend.getOutputStream().write(...); // APIへ送るXMLデータをOutputStreamへ書く

connSend.connect(); // APIへ接続して、データを送る
if(connSend.getResponseCode() != HttpURLConnection.HTTP_OK) {
    throw new Exception("エラー");
}

... conn.getInputStream(); // APIから貰うデータをInputStreamから呼んで、業務ロジックを行う

connSend.disconnect(); // 接続を切る
```

3.API一覧

3.5.2.2. JSON形式でリクエストした場合

```
// APIへJSONドキュメントを渡す方法
URL urlSend = new URL("https://dummy.sagawa-exp.co.jp/rest/insatsumodule/shipping/json"); // 送り状発行APIのURL
URLConnection connSend = (URLConnection)urlSend.openConnection();
connSend.setDoOutput(true); // APIへ送るデータ
connSend.setDoInput(true); // APIから貰うデータ
connSend.setRequestMethod("POST"); // POSTが必須
connSend.setRequestProperty("Content-Type","application/json"); // 送るデータの形式はjson

connSend.getOutputStream().write(...); // APIへ送るJSONデータをOutputStreamへ書く

connSend.connect(); // APIへ接続して、データを送る
if(connSend.getResponseCode() != HttpURLConnection.HTTP_OK) {
    throw new Exception("エラー");
}

... conn.getInputStream(); // APIから貰うデータをInputStreamから呼んで、業務ロジックを行う

connSend.disconnect(); // 接続を切る
```

3.API一覧

3.5.2.3. XML形式でリクエストした場合(即時発行APIを利用する場合のサンプル)

```
// APIへXMLドキュメントを渡す方法
URL urlSend = new URL("https://dummy.sagawa-exp.co.jp/rest/insatsumodule/sokuji/xml"); // 即時発行APIのURL
URLConnection connSend = (URLConnection)urlSend.openConnection();
connSend.setDoOutput(true); // APIへ送るデータ
connSend.setDoInput(true); // APIから貰うデータ
connSend.setRequestMethod("POST"); // POSTが必須
connSend.setRequestProperty("Content-Type","application/xml"); // 送るデータの形式はXML

connSend.getOutputStream().write(...); // APIへ送るXMLデータをOutputStreamへ書く

connSend.connect(); // APIへ接続して、データを送る
if(connSend.getResponseCode() != HttpURLConnection.HTTP_OK) {
    throw new Exception("エラー");
}

... conn.getInputStream(); // APIから貰うデータをInputStreamから呼んで、業務ロジックを行う

connSend.disconnect(); // 接続を切る

String urlString = ... ; //APIレスポンスデータよりurlタグ要素を取得
URL url = new URL(urlString);
... // 取得したurlを確認、リトライ処理を追加（URL返却のタイミングではPDF生成が完了していない可能性があります）
```

3.API一覧

3.5.2.4. JSON形式でリクエストした場合(即時発行APIを利用する場合のサンプル)

```
// APIへJSONドキュメントを渡す方法
URL urlSend = new URL("https://dummy.sagawa-exp.co.jp/rest/insatsumodule/sokuji/json"); // 即時発行APIのURL
URLConnection connSend = (URLConnection)urlSend.openConnection();
connSend.setDoOutput(true); // APIへ送るデータ
connSend.setDoInput(true); // APIから貰うデータ
connSend.setRequestMethod("POST"); // POSTが必須
connSend.setRequestProperty("Content-Type","application/json"); // 送るデータの形式はjson

connSend.getOutputStream().write(...); // APIへ送るJSONデータをOutputStreamへ書く

connSend.connect(); // APIへ接続して、データを送る
if(connSend.getResponseCode() != HttpURLConnection.HTTP_OK) {
    throw new Exception("エラー");
}

... conn.getInputStream(); // APIから貰うデータをInputStreamから呼んで、業務ロジックを行う

connSend.disconnect(); // 接続を切る

String urlString = ... ; //APIレスポンスデータよりurl要素を取得
URL url = new URL(urlString);
... // 取得したurlを確認、リトライ処理を追加（URL返却のタイミングではPDF生成が完了していない可能性があります）
```

4.インターフェース定義

4.1.送り状発行API I-F

リクエスト

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	入力例
送り状発行 リクエスト	1	1	送り状発行リクエスト	-	shippingRequest		XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	ユーザー認証	-	customerAuth			◎	
	1-1-1	3	カスタマーID		customerId	半角文字	50	◎	12345678
	1-1-2	3	ログインパスワード		loginPassword	半角文字	100	◎	dx3jXMk542T+uCvIldpUQ==
	1-2	2	配送会社コード		deliveryCode	半角文字	4		0001
	1-3	2	送り状発行依頼フラグ		printOutFlg	半角数字	1	◎	1
	1-4	2	送り状コード		okuriCode	半角文字	20	○	A501
	1-5	2	出力レベル		outputLevel	半角数字	3	◎	000
	1-6	2	下敷画像表示フラグ		backLayerFlg	半角数字	1		1
	1-7	2	出荷情報リスト	-	printDataList			◎	
	1-7-1	3	出荷情報明細	-	printDataDetail		出荷情報が複数ある場合、繰り返し指定可能	◎	
	1-7-1-1	4	配送個口数		haisoKosu	半角数字	2	◎	1
	1-7-1-2	4	管理番号		userManagerNumber	半角文字	16	◎	20220801092404
	1-7-1-3	4	顧客コード		kokyakuCode	半角数字	12	◎	999999999999
	1-7-1-4	4	届先住所1		otodokeAdd1	指定なし	25	◎	東京都江東区新砂
	1-7-1-5	4	届先住所2		otodokeAdd2	指定なし	25		2-2-8
	1-7-1-6	4	届先住所3		otodokeAdd3	指定なし	25		佐川急便株式会社
	1-7-1-7	4	届先氏名1		otodokeNm1	指定なし	25	◎	飛脚 花子
	1-7-1-8	4	届先氏名2		otodokeNm2	指定なし	25		
	1-7-1-9	4	届先郵便番号		otodokeYubin	半角数字	7	◎	1360075
	1-7-1-10	4	届先電話番号		otodokeTel	半角文字	20	◎	00-0000-0000
	1-7-1-11	4	届先メールアドレス		otodokeMailAddress	半角文字	320		
	1-7-1-12	4	依頼主指定フラグ		iraiPrintFlg	半角数字	1	◎	1

4.インターフェース定義

4.1.送り状発行API I-F

1-7-1-13	4	依頼主住所1	iraiAdd1	指定なし	25	依頼主指定フラグが1の場合必須 半角/全角問わず、25文字まで印字可能	○	京都府京都市南区上鳥羽角田町
1-7-1-14	4	依頼主住所2	iraiAdd2	指定なし	25	半角/全角問わず、25文字まで印字可能		6 8
1-7-1-15	4	依頼主住所3	iraiAdd3	指定なし	25	半角/全角問わず、25文字まで印字可能		京都本社
1-7-1-16	4	依頼主氏名1	iraiNm1	指定なし	25	依頼主指定フラグが1の場合必須 半角/全角問わず、25文字まで印字可能	○	佐川 太郎
1-7-1-17	4	依頼主氏名2	iraiNm2	指定なし	25	半角/全角問わず、25文字まで印字可能		
1-7-1-18	4	依頼主郵便番号	iraiYubin	半角数字	7	依頼主指定フラグが1の場合必須 ハイフンなし	○	6018104
1-7-1-19	4	依頼主電話番号	iraiTel	半角文字	20	依頼主指定フラグが1の場合必須 数字またはハイフン	○	00-0000-0000
1-7-1-20	4	依頼主メールアドレス	iraiMailAddress	半角文字	320			
1-7-1-21	4	発送日	shippingDate	半角文字	8	yyyyMMdd形式で設定		20220801
1-7-1-22	4	記事1	kiji1	指定なし	32	半角/全角問わず、32文字まで印字可能		スポーツ用品
1-7-1-23	4	記事2	kiji2	指定なし	32	半角/全角問わず、32文字まで印字可能		宅配BOXへ入れてください
1-7-1-24	4	記事3	kiji3	指定なし	32	半角/全角問わず、32文字まで印字可能		
1-7-1-25	4	記事4	kiji4	指定なし	32	半角/全角問わず、32文字まで印字可能		
1-7-1-26	4	記事5	kiji5	指定なし	32	半角/全角問わず、32文字まで印字可能		
1-7-1-27	4	記事6	kiji6	指定なし	32	半角/全角問わず、32文字まで印字可能		
1-7-1-28	4	便種コード	binsyuCode	半角数字	3	5.1.パラメータコード定義参照	◎	030
1-7-1-29	4	代金引換フラグ	daibikiFlg	半角数字	1	0 - 通常出荷の送り状を発行 1 - 代金引換の送り状を発行	◎	0
1-7-1-30	4	代引支払方法区分	daibikiType	半角数字	1	代金引換時、受取人の支払方法を設定 5.1.パラメータコード定義書参照 空白の場合はシステムに登録されている値が代用されます		
1-7-1-31	4	配達指定日	shiteiDate	半角文字	8	yyyyMMdd形式で設定		20220805
1-7-1-32	4	配達時間指定コード	shiteiTimeCode	半角文字	2	5.1.パラメータコード定義参照		12
1-7-1-33	4	代引金額	daibikiKingaku	半角数字	8	佐川急便が荷物配送時に回収する代引金額(消費税込み) 代金引換フラグが1の場合必須 代金引換フラグが0/null/空白の場合、反映されません	○	3600
1-7-1-34	4	代引消費税	daibikiTax	半角数字	8	代引金額に含まれている消費税額 代金引換フラグが1の場合必須 代金引換フラグが0/null/空白の場合、反映されない	○	266
1-7-1-35	4	重量1	weight1	半角文字	3	佐川急便が荷物のサイズを登録する際のコード null/空白の場合はシステムで自動的に付与される 5.1.パラメータコード定義参照		
1-7-1-36	4	重量2	weight2	半角文字	3	佐川急便が荷物のサイズを登録する際のコード null/空白の場合はシステムで自動的に付与される 5.1.パラメータコード定義参照		
1-7-1-37	4	シール1	careSeal1	半角文字	3	5.1.パラメータコード定義参照		012
1-7-1-38	4	シール2	careSeal2	半角文字	3	5.1.パラメータコード定義参照		013
1-7-1-39	4	シール3	careSeal3	半角文字	3	5.1.パラメータコード定義参照		011

4.インターフェース定義

4.1.送り状発行API I-F

	1-7-1-40	4	保険金額	hokenKingaku	半角数字	8	輸送中の盗難・破損などによる貨物の損害を補償する一輸送 単位の運送保険 空白/nullの場合は、反映されない		
	1-7-1-41	4	営止めフラグ	eidomeFlg	半角数字	1	荷物を営業所止めにするか設定 営業所止め先は営止めコードを参照 0/null/空白 - 営業所止めにしない 1 - 営業所止めにする	0	
	1-7-1-42	4	営業所コード	depotCode	半角文字	10	営止めフラグが1の場合、営業コードに紐づく営業所へ荷物 の営業止めを行う 営止めコードに記載がない場合はお届け先住所へ届ける担当 営業所を営業所止め先にする	3029	
	1-7-1-43	4	マーク	mark	指定なし	5	使用不可		
	1-7-1-44	4	元着コード	motoChakuCode	半角数字	1	配送料金の支払い方法を設定 5.1.パラメータコード定義参照 ※元払いのみ設定可能	0	

4.インターフェース定義

4.1.送り状発行API I-F

レスポンス

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	出力例
送り状発行 レスポンス	1	1	送り状発行レスポンス	- shippingResponse			XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	送り状発行依頼フラグ	printOutFlg	半角数字	1	リクエスト時の値を返却	◎	1
	1-2	2	送り状コード	okuriCode	半角文字	20	リクエスト時の値を返却	◎	A501
	1-3	2	出力レベル	outputLevel	半角数字	3	リクエスト時の値を返却	◎	000
	1-4	2	処理結果コード	resultCode	半角文字	8	リクエスト全体の処理結果を返却	◎	S0-0001
	1-5	2	発行受付ID	printRequestId	半角文字	255	送付状発行フラグが1、かつリクエスト全体の処理結果が正常の場合のみ返却		2064-1
	1-6	2	出荷情報リスト	- printDataList			リクエストの出荷情報リストに1つでもエラーが存在する場合		
	1-6-1	3	出荷情報明細	- printDataDetail					
	1-6-1-1	4	管理番号	userManageNumber	半角文字	16	リクエスト時の値を返却		20220801092404
	1-6-1-2	4	結果コードリスト	- resultCodeList					
	1-6-1-2-1	5	処理結果コード	resultCode	半角文字	8	処理結果コードが複数ある場合繰り返し返却		S0-0001
	1-6-1-3	4	営業所情報	- depotInfo					
	1-6-1-3-1	5	営業所コード	depotCode	半角文字	10			
	1-6-1-3-2	5	営業所名	depotName	指定なし	60			
	1-6-1-3-3	5	営業所電話番号	depotTel	半角文字	20			
	1-6-1-4	4	問合番号リスト	- shippingNumberList			送り状発行フラグが1、かつリクエスト全体の処理結果が正常の場合のみ返却		
	1-6-1-4-1	5	問合番号	shippingNumber	半角文字	50	問合番号を付与		999999999999

4.インターフェース定義

4.1.送り状発行API I-F例

4.1.1. XML形式の場合

リクエスト

```
<?xml version="1.0" encoding="UTF-8"?>
<shippingRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword><![CDATA[dx3jXmK542T+UjCvIldpUQ==]]>
    </loginPassword>
  </customerAuth>
  <deliveryCode></deliveryCode>
  <printOutFlg>0</printOutFlg>
  <okuriCode>A501</okuriCode>
  <outputLevel>000</outputLevel>
  <backLayerFlg>1</backLayerFlg>
  <printDataList>
    <printDataDetail>
      <haisoKosu>1</haisoKosu>
      <userManagerNumber>20220801092404</userManagerNumber>
      <kokyakuCode>999999999999</kokyakuCode>
      <otodokeAdd1>東京都江東区新砂</otodokeAdd1>
      <otodokeAdd2>2 - 2 - 8</otodokeAdd2>
      <otodokeAdd3>佐川急便株式会社</otodokeAdd3>
      <otodokeNm1>飛脚 花子</otodokeNm1>
      <otodokeNm2 />
      <otodokeYubin>1360075</otodokeYubin>
      <otodokeTel>00-0000-0000</otodokeTel>
      <otodokeMailAddress>hikyaku.hanako@sagawa-exp.co.jp
      </otodokeMailAddress>
      <iraiPrintFlg>1</iraiPrintFlg>
      <iraiAdd1>京都府京都市南区上鳥羽角田町</iraiAdd1>
      <iraiAdd2>6 8</iraiAdd2>
      <iraiAdd3>京都本社</iraiAdd3>
      <iraiNm1>佐川 太郎</iraiNm1>
      <iraiNm2 />
      <iraiYubin>6018104</iraiYubin>
      <iraiTel>00-0000-0000</iraiTel>
      <iraiMailAddress>
      </iraiMailAddress>
      <shippingDate />
      <kiji1>スポーツ用品</kiji1>
      <kiji2>宅配BOXへ入れてください</kiji2>
      <kiji3 />
      <kiji4 />
      <kiji5 />
      <kiji6 />
      <binsyuCode>030</binsyuCode>
```

```
<daibikiFlg>0</daibikiFlg>
<daibikiType />
<shiteiDate>20220805</shiteiDate>
<shiteiTimeCode/>
<daibikiKingaku/>
<daibikiTax/>
<weight1 />
<weight2 />
<careSeal1>012</careSeal1>
<careSeal2>013</careSeal2>
<careSeal3>011</careSeal3>
<hokenKingaku/>
<eidomeFlg />
<depotCode>3029</depotCode>
<mark />
<motoChakuCode />
</printDataDetail>
</printDataList>
</shippingRequest>
```

レスポンス 正常終了

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<shippingResponse>
  <printOutFlg>1</printOutFlg>
  <okuriCode>A501</okuriCode>
  <outputLevel>000</outputLevel>
  <resultCode>S0-0001</resultCode>
  <printRequestId>2064-1</printRequestId>
  <printDataList>
    <printDataDetail>
      <userManagerNumber>20220801092404</userManagerNumber>
      <resultCodeList>
        <resultCode>S0-0001</resultCode>
      </resultCodeList>
      <shippingNumberList>
        <shippingNumber>999999999999</shippingNumber>
      </shippingNumberList>
    </printDataDetail>
  </printDataList>
</shippingResponse>
```

レスポンス エラー有

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<shippingResponse>
  <printOutFlg>1</printOutFlg>
  <okuriCode>A501</okuriCode>
  <outputLevel>000</outputLevel>
  <resultCode>E8-0001</resultCode>
  <printRequestId>2064-1</printRequestId>
  <printDataList>
    <printDataDetail>
      <userManagerNumber>20220801092404</userManagerNumber>
      <resultCodeList>
        <resultCode>E1-0001</resultCode>
        <resultCode>E1-0002</resultCode>
      </resultCodeList>
    </printDataDetail>
  </printDataList>
</shippingResponse>
```

4.インターフェース定義

4.1.送り状発行API I-F例

4.1.2. JSON形式の場合

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"dx3jXMk542T+UcVldpUQ=="
  },
  "printOutFlg":"0",
  "okuriCode":"A501",
  "outputLevel":"000",
  "backLayerFlg":"1",
  "deliveryCode": "",
  "printDataList":{
    "printDataDetail":[
      {
        "haisoKosu":"1",
        "userManagerNumber":"20220801092404",
        "kokyakuCode":"999999999999",
        "otodokeAdd1":"東京都江東区新砂",
        "otodokeAdd2":"2 - 2 - 8",
        "otodokeAdd3":"佐川急便株式会社",
        "otodokeNm1":"飛脚花子",
        "otodokeNm2":"","",
        "otodokeYubin":"1360075",
        "otodokeTel":"00-0000-0000",
        "otodokeMailAddress":"hikyaku.hanako@sagawa.co.jp",
        "iraiPrintFlg":"1",
        "iraiAdd1":"京都府京都市南区上烏羽角田町",
        "iraiAdd2":"6 8",
        "iraiAdd3":"京都本社",
        "iraiNm1":"佐川太郎",
        "iraiNm2":"","",
        "iraiYubin":"6018104",
        "iraiTel":"00-0000-0000",
        "iraiMailAddress":"","",
        "shippingDate":"","",
        "kiji1":"スポーツ用品",
        "kiji2":"宅配BOXへ入れてください",
        "kiji3":"","",
        "kiji4":"","",
        "kiji5":"","",
        "kiji6":"","",
        "binsyuCode":"030",
        "daibikiFlg":"0",
        "daibikiType":"","",
        "shiteiDate":"20220805",
        "shiteiTimeCode":"","",
        "daibikiKingaku":"","",
        "daibikiTax":"","",

```

```
      "weight1":"","",
      "weight2":"","",
      "careSeal1":"012",
      "careSeal2":"013",
      "careSeal3":"011",
      "hokenKingaku":"","",
      "eidomeFlg":"","",
      "depotCode":"3029",
      "mark":"","",
      "motoChakuCode":"","",
    ]
  }
}
```

レスポンス 正常終了

```
{
  "printOutFlg":"1",
  "okuriCode":"A501",
  "outputLevel":"000",
  "resultCode":"S0-0001",
  "printRequestId":"2064-1",
  "printDataList":{
    "printDataDetail":[
      {
        "userManagerNumber":"20220801092404",
        "resultCodeList":{
          "resultCode":[
            "S0-0001"
          ]
        },
        "shippingNumberList":{
          "shippingNumber":[
            "999999999999"
          ]
        }
      ]
    ]
  }
}
```

レスポンス エラー有

```
{
  "printOutFlg":"1",
  "okuriCode":"A501",
  "outputLevel":"000",
  "resultCode":"E8-0001",
  "printRequestId":"2064-1",
  "printDataList":{
    "printDataDetail":[
      {
        "userManagerNumber":"20220801092404",
        "resultCodeList":{
          "resultCode":[
            "E1-0001",
            "E1-0002"
          ]
        }
      ]
    ]
  }
}
```

4.インターフェース定義

4.2.即時発行API I-F

リクエスト

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	入力例
即時発行 リクエスト	1	1	即時発行リクエスト	-	sokujiRequest		XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	ユーザー認証	-	customerAuth			◎	
	1-1-1	3	カスタマーID	customerId	半角文字	50		◎	12345678
	1-1-2	3	ログインパスワード	loginPassword	半角文字	100		◎	dx3jXMK542T+tJCvIldpUQ==
	1-2	2	配送会社コード	deliveryCode	半角文字	4	下記「佐川急便」のコードを設定 0001/null/空白 - 佐川急便		0001
	1-3	2	送り状発行依頼フラグ	printOutFlg	半角数字	1	0 - (確認機能) エラー精査のみ行う 1 - (発行依頼機能) エラー精査後、出荷情報が全て正常であれば送り状発行処理を行う	◎	1
	1-4	2	送り状コード	okuriCode	半角文字	20	送り状の帳票コードを設定 5.1.パラメータコード定義書参照 送り状発行依頼フラグが1の場合、指定必須	○	A501
	1-5	2	出力レベル	outputLevel	半角数字	3	エラー判定の精査範囲を設定 5.1.パラメータコード定義書参照	◎	000
	1-6	2	下敷画像表示フラグ	backLayerFlg	半角数字	1	帳票の背景画像をPDFに表示させるかのフラグ 0/null/空白 - 非表示 1 - 表示		1
	1-7	2	出荷情報リスト	-	printDataList			◎	
	1-7-1	3	出荷情報明細	-	printDataDetail		出荷情報が複数ある場合、繰り返し指定可能	◎	
	1-7-1-1	4	配送個口数	haisoKosu	半角数字	2	発送個口数 指定分送り状発行	◎	1
	1-7-1-2	4	管理番号	userManageNumber	半角文字	16	利用者側で管理する一意なコード	◎	20220801092404
	1-7-1-3	4	顧客コード	kokyakuCode	半角数字	12	佐川急便との契約コード チェックデジットを含む	◎	999999999999
	1-7-1-4	4	屈先住所1	otodokeAdd1	指定なし	25	半角/全角問わず、25文字まで印字可能	◎	東京都江東区新砂
	1-7-1-5	4	屈先住所2	otodokeAdd2	指定なし	25	半角/全角問わず、25文字まで印字可能		2 - 2 - 8
	1-7-1-6	4	屈先住所3	otodokeAdd3	指定なし	25	半角/全角問わず、25文字まで印字可能		佐川急便株式会社
	1-7-1-7	4	屈先氏名1	otodokeNm1	指定なし	25	半角/全角問わず、25文字まで印字可能	◎	飛脚 花子
	1-7-1-8	4	屈先氏名2	otodokeNm2	指定なし	25	半角/全角問わず、25文字まで印字可能		
	1-7-1-9	4	屈先郵便番号	otodokeYubin	半角数字	7	ハイフン無し	◎	1360075
	1-7-1-10	4	屈先電話番号	otodokeTel	半角文字	20	数字またはハイフン	◎	00-0000-0000
	1-7-1-11	4	屈先メールアドレス	otodokeMailAddress	半角文字	320			
	1-7-1-12	4	依頼主指定フラグ	iraiPrintFlg	半角数字	1	送り状の依頼主項目の印字を、顧客コードに紐づく出荷場にするか、下記の依頼主情報を記載するか設定 0 - 顧客コードに紐づく出荷場情報を印字 1 - 下記の依頼主情報を参照	◎	1

4.インターフェース定義

4.2.即時発行API I-F

1-7-1-13	4	依頼主住所1	iraiAdd1	指定なし	25	依頼主指定フラグが1の場合必須 半角/全角問わず、25文字まで印字可能	○	京都府京都市南区上鳥羽角田町
1-7-1-14	4	依頼主住所2	iraiAdd2	指定なし	25	半角/全角問わず、25文字まで印字可能		6 8
1-7-1-15	4	依頼主住所3	iraiAdd3	指定なし	25	半角/全角問わず、25文字まで印字可能		京都本社
1-7-1-16	4	依頼主氏名1	iraiNm1	指定なし	25	依頼主指定フラグが1の場合必須 半角/全角問わず、25文字まで印字可能	○	佐川 太郎
1-7-1-17	4	依頼主氏名2	iraiNm2	指定なし	25	半角/全角問わず、25文字まで印字可能		
1-7-1-18	4	依頼主郵便番号	iraiYubin	半角数字	7	依頼主指定フラグが1の場合必須 ハイフンなし	○	6018104
1-7-1-19	4	依頼主電話番号	iraiTel	半角文字	20	依頼主指定フラグが1の場合必須 数字またはハイフン	○	00-0000-0000
1-7-1-20	4	依頼主メールアドレス	iraiMailAddress	半角文字	320			
1-7-1-21	4	発送日	shippingDate	半角文字	8	yyyyMMdd形式で設定		20220801
1-7-1-22	4	記事1	kiji1	指定なし	32	半角/全角問わず、32文字まで印字可能		スポーツ用品
1-7-1-23	4	記事2	kiji2	指定なし	32	半角/全角問わず、32文字まで印字可能		宅配BOXへ入れてください
1-7-1-24	4	記事3	kiji3	指定なし	32	半角/全角問わず、32文字まで印字可能		
1-7-1-25	4	記事4	kiji4	指定なし	32	半角/全角問わず、32文字まで印字可能		
1-7-1-26	4	記事5	kiji5	指定なし	32	半角/全角問わず、32文字まで印字可能		
1-7-1-27	4	記事6	kiji6	指定なし	32	半角/全角問わず、32文字まで印字可能		
1-7-1-28	4	便種コード	binsyuCode	半角数字	3	5.1.パラメータコード定義参照	◎	030
1-7-1-29	4	代金引換フラグ	daibikiFlg	半角数字	1	0 - 通常出荷の送り状を発行 1 - 代金引換の送り状を発行	◎	0
1-7-1-30	4	代引支払方法区分	daibikiType	半角数字	1	代金引換時、受取人の支払方法を設定 5.1.パラメータコード定義書参照 空白の場合はシステムに登録されている値が代用されます		
1-7-1-31	4	配達指定日	shiteiDate	半角文字	8	yyyyMMdd形式で設定		20220805
1-7-1-32	4	配達時間指定コード	shiteiTimeCode	半角文字	2	5.1.パラメータコード定義参照		12
1-7-1-33	4	代引金額	daibikiKingaku	半角数字	8	佐川急便が荷物配送時に回収する代引金額(消費税込み) 代金引換フラグが1の場合必須 代金引換フラグが0/null/空白の場合、反映されません	○	3600
1-7-1-34	4	代引消費税	daibikiTax	半角数字	8	代引金額に含まれている消費税額 代金引換フラグが1の場合必須 代金引換フラグが0/null/空白の場合、反映されない	○	266
1-7-1-35	4	重量1	weight1	半角文字	3	佐川急便が荷物のサイズを登録する際のコード null/空白の場合はシステムで自動的に付与される 5.1.パラメータコード定義参照		
1-7-1-36	4	重量2	weight2	半角文字	3	佐川急便が荷物のサイズを登録する際のコード null/空白の場合はシステムで自動的に付与される 5.1.パラメータコード定義参照		
1-7-1-37	4	シール1	careSeal1	半角文字	3	5.1.パラメータコード定義参照		012
1-7-1-38	4	シール2	careSeal2	半角文字	3	5.1.パラメータコード定義参照		013
1-7-1-39	4	シール3	careSeal3	半角文字	3	5.1.パラメータコード定義参照		011

4.インターフェース定義

4.2.即時発行API I-F

	1-7-1-40	4	保険金額	hokenKingaku	半角数字	8	輸送中の盗難・破損などによる貨物の損害を補償する一輸送 単位の運送保険 空白/nullの場合は、反映されない		
	1-7-1-41	4	営止めフラグ	eidomeFlg	半角数字	1	荷物を営業所止めにするか設定 営業所止め先は営止めコードを参照 0/null/空白 - 営業所止めにしない 1 - 営業所止めにする	0	
	1-7-1-42	4	営業所コード	depotCode	半角文字	10	営止めフラグが1の場合、営業コードに紐づく営業所へ荷物 の営業止めを行う 営止めコードに記載がない場合はお届け先住所へ届ける担当 営業所を営業所止め先にする	3029	
	1-7-1-43	4	マーク	mark	指定なし	5	使用不可		
	1-7-1-44	4	元着コード	motoChakuCode	半角数字	1	配送料金の支払い方法を設定 5.1.パラメータコード定義参照 ※元払いのみ設定可能	0	

4.インターフェース定義

4.2.即時発行API I-F

レスポンス

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	出力例
即時発行 レスポンス	1	1	即時発行レスポンス	sokujiResponse			XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	送り状発行依頼フラグ	printOutFlg	半角数字	1	リクエスト時の値を返却	●	1
	1-2	2	送り状コード	okuriCode	半角文字	20	リクエスト時の値を返却	●	A501
	1-3	2	出力レベル	outputLevel	半角数字	3	リクエスト時の値を返却	●	000
	1-4	2	処理結果コード	resultCode	半角文字	8	リクエスト全体の処理結果を返却	●	S0-0001
	1-5	2	取得URL	url	半角文字	2000	送り状PDFのダウンロード先URL		https://XXXXXXXXXXXXXXXXX
	1-6	2	出荷情報リスト	printDataList					
	1-6-1	3	出荷情報明細	printDataDetail					
	1-6-1-1	4	管理番号	userManagerNumber	半角文字	16	リクエスト時の値を返却		20220801092404
	1-6-1-2	4	結果コードリスト	resultCodeList					
	1-6-1-2-1	5	処理結果コード	resultCode	半角文字	8	処理結果コードが複数ある場合繰り返し返却		S0-0001
	1-6-1-3	4	営業所情報	depotInfo					
	1-6-1-3-1	5	営業所コード	depotCode	半角文字	10			
	1-6-1-3-2	5	営業所名	depotName	指定なし	60			
	1-6-1-3-3	5	営業所電話番号	depotTel	半角文字	20			
	1-6-1-4	4	問合番号リスト	shippingNumberList			送り状発行フラグが1、かつリクエスト全体の処理結果が正常の場合のみ返却		
	1-6-1-4-1	5	問合番号	shippingNumber	半角文字	50	問合番号を付与		999999999999

4.インターフェース定義

4.2.即時発行API I-F例

4.1.1. XML形式の場合

リクエスト

```
<?xml version="1.0" encoding="UTF-8"?>
<sokujiRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword><![CDATA[dx3jXmK542T+UjCvIldpUQ==]]>
    </loginPassword>
  </customerAuth>
  <deliveryCode></deliveryCode>
  <printOutFlg>0</printOutFlg>
  <okuriCode>A501</okuriCode>
  <outputLevel>000</outputLevel>
  <backLayerFlg>1</backLayerFlg>
  <printDataList>
    <printDataDetail>
      <haisoKosu>1</haisoKosu>
      <userManagerNumber>20220801092404</userManagerNumber>
      <kokyakuCode>999999999999</kokyakuCode>
      <otodokeAdd1>東京都江東区新砂</otodokeAdd1>
      <otodokeAdd2>2 - 2 - 8</otodokeAdd2>
      <otodokeAdd3>佐川急便株式会社</otodokeAdd3>
      <otodokeNm1>飛脚 花子</otodokeNm1>
      <otodokeNm2 />
      <otodokeYubin>1360075</otodokeYubin>
      <otodokeTel>00-0000-0000</otodokeTel>
      <otodokeMailAddress>hikyaku.hanako@sagawa-exp.co.jp
      </otodokeMailAddress>
      <iraiPrintFlg>1</iraiPrintFlg>
      <iraiAdd1>京都府京都市南区上鳥羽角田町</iraiAdd1>
      <iraiAdd2>6 8</iraiAdd2>
      <iraiAdd3>京都本社</iraiAdd3>
      <iraiNm1>佐川 太郎</iraiNm1>
      <iraiNm2 />
      <iraiYubin>6018104</iraiYubin>
      <iraiTel>00-0000-0000</iraiTel>
      <iraiMailAddress>
      </iraiMailAddress>
      <shippingDate />
      <kiji1>スポーツ用品</kiji1>
      <kiji2>宅配BOXへ入れてください</kiji2>
      <kiji3 />
      <kiji4 />
      <kiji5 />
      <kiji6 />
      <binsyuCode>030</binsyuCode>
```

```
<daibikiFlg>0</daibikiFlg>
<daibikiType />
<shiteiDate>20220805</shiteiDate>
<shiteiTimeCode/>
<daibikiKingaku/>
<daibikiTax/>
<weight1 />
<weight2 />
<careSeal1>012</careSeal1>
<careSeal2>013</careSeal2>
<careSeal3>011</careSeal3>
<hokenKingaku/>
<eidomeFlg />
<depotCode>3029</depotCode>
<mark />
<motoChakuCode />
</printDataDetail>
</printDataList>
</sokujiRequest>
```

レスポンス 正常終了

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<sokujiResponse>
  <printOutFlg>1</printOutFlg>
  <okuriCode>A501</okuriCode>
  <outputLevel>000</outputLevel>
  <resultCode>S0-0001</resultCode>
  <url>https://dummy.sagawa-exp.co.jp/</url>
  <printDataList>
    <printDataDetail>
      <userManagerNumber>20220801092404</userManagerNumber>
      <resultCodeList>
        <resultCode>S0-0001</resultCode>
      </resultCodeList>
      <shippingNumberList>
        <shippingNumber>999999999999</shippingNumber>
      </shippingNumberList>
    </printDataDetail>
  </printDataList>
</sokujiResponse>
```

レスポンス エラー有

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<sokujiResponse>
  <printOutFlg>1</printOutFlg>
  <okuriCode>A501</okuriCode>
  <outputLevel>000</outputLevel>
  <resultCode>E8-0001</resultCode>
  <printDataList>
    <printDataDetail>
      <userManagerNumber>20220801092404</userManagerNumber>
      <resultCodeList>
        <resultCode>E1-0001</resultCode>
        <resultCode>E1-0002</resultCode>
      </resultCodeList>
    </printDataDetail>
  </printDataList>
</sokujiResponse>
```

4.インターフェース定義

4.2.即時発行API I-F例

4.1.2. JSON形式の場合

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"dx3jXmk542T+UcVldpUQ=="
  },
  "printOutFlg":"0",
  "okuriCode":"A501",
  "outputLevel":"000",
  "backLayerFlg":"1",
  "deliveryCode":"",
  "printDataList":{
    "printDataDetail":[
      {
        "haisoKosu":"1",
        "userManagerNumber":"20220801092404",
        "kokyakuCode":"999999999999",
        "otodokeAdd1":"東京都江東区新砂",
        "otodokeAdd2":"2 - 2 - 8",
        "otodokeAdd3":"佐川急便株式会社",
        "otodokeNm1":"飛脚花子",
        "otodokeNm2":"",
        "otodokeYubin":"1360075",
        "otodokeTel":"00-0000-0000",
        "otodokeMailAddress":"hikyaku.hanako@sagawa.co.jp",
        "iraiPrintFlg":"1",
        "iraiAdd1":"京都府京都市南区上烏羽角田町",
        "iraiAdd2":"6 8",
        "iraiAdd3":"京都本社",
        "iraiNm1":"佐川太郎",
        "iraiNm2":"",
        "iraiYubin":"6018104",
        "iraiTel":"00-0000-0000",
        "iraiMailAddress":"",
        "shippingDate":"",
        "kiji1":"スポーツ用品",
        "kiji2":"宅配BOXへ入れてください",
        "kiji3":"",
        "kiji4":"",
        "kiji5":"",
        "kiji6":"",
        "binsyuCode":"030",
        "daibikiFlg":"0",
        "daibikiType":"",
        "shiteiDate":"20220805",
        "shiteiTimeCode":"",
        "daibikiKingaku":"",
        "daibikiTax":""
      }
    ]
  }
}
```

```

    "weight1":"","
    "weight2":"","
    "careSeal1":"012",
    "careSeal2":"013",
    "careSeal3":"011",
    "hokenKingaku":"","
    "eidomeFlg":"","
    "depotCode":"3029",
    "mark":"","
    "motoChakuCode":"","
  }
}
}
```

レスポンス 正常終了

```
{
  "printOutFlg":"1",
  "okuriCode":"A501",
  "outputLevel":"000",
  "resultCode":"S0-0001",
  "url":"https://dummy.sagawa-exp.co.jp/"
  "printDataList":{
    "printDataDetail":[
      {
        "userManagerNumber":"20220801092404",
        "resultCodeList":{
          "resultCode":[
            "S0-0001"
          ]
        },
        "shippingNumberList":{
          "shippingNumber":[
            "999999999999"
          ]
        }
      }
    ]
  }
}
```

レスポンス エラー有

```
{
  "printOutFlg":"1",
  "okuriCode":"A501",
  "outputLevel":"000",
  "resultCode":"E8-0001",
  "printDataList":{
    "printDataDetail":[
      {
        "userManagerNumber":"20220801092404",
        "resultCodeList":{
          "resultCode":[
            "E1-0001",
            "E1-0002"
          ]
        }
      }
    ]
  }
}
```

4.インターフェース定義

4.3.ファイル存在確認API I-F

リクエスト

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	入力例
ファイル存在確認 リクエスト	1	1	ファイル存在確認リクエスト	-	checkFileRequest		XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	ユーザー 認証	-	customerAuth			◎	
	1-1-1	3	カスタマーID		customerId	半角文字	50	◎	12345678
	1-1-2	3	ログインパスワード		loginPassword	半角文字	100	◎	dx3jXMk542T+tJCvIldpUQ==
	1-2	2	発行受付IDリスト	-	printRequestIdList			◎	
	1-2-1	3	発行受付ID		printRequestId	半角文字	255	◎	2064-1

レスポンス

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	出力例
ファイル存在確認 レスポンス	1	1	ファイル存在確認レスポンス	-	checkFileResponse		XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	処理結果コード		resultCode	半角文字	8	◎	S0-0001
	1-2	2	出荷情報リスト	-	printDataList				
	1-2-1	3	出荷情報明細	-	printDataDetail		発行受付IDが複数ある場合は繰り返し返却		
	1-2-1-1	4	発行受付ID		printRequestId	半角文字	255		2064-1
	1-2-1-2	4	データ発行日		createDate	半角文字	20		2022/08/01 18:22:24
	1-2-1-3	4	処理結果コード		resultCode	半角文字	8		S0-0001
	1-2-1-4	4	取得URL		url	半角文字	2000		https://XXXXXXXXXXXXXXXXX

4.インターフェース定義

4.3.ファイル存在確認API I-F例

4.2.1. XML形式の場合

リクエスト

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<checkFileRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword>dx3jXmk542T+UjCvIldpUQ==</loginPassword>
  </customerAuth>
  <printRequestIdList>
    <printRequestId>2064-1</printRequestId>
  </printRequestIdList>
</checkFileRequest>
```

レスポンス 正常終了

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<checkFileResponse>
  <resultCode>S0-0001</resultCode>
  <printDataList>
    <printDataDetail>
      <printRequestId>2064-1</printRequestId>
      <createDate>2022/08/01 18:22:24</createDate>
      <resultCode>S0-0001</resultCode>
      <url>https://dummy.sagawa-exp.co.jp/</url>
    </printDataDetail>
  </printDataList>
</checkFileResponse>
```

レスポンス エラー有（ファイル取得有効期限切れ）

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<checkFileResponse>
  <resultCode>E8-0001</resultCode>
  <printDataList>
    <printDataDetail>
      <printRequestId>11-1</printRequestId>
      <createDate>2022/07/21 15:53:48</createDate>
      <resultCode>E2-0013</resultCode>
    </printDataDetail>
  </printDataList>
</checkFileResponse>
```

4.インターフェース定義

4.3.ファイル存在確認API I-F例

4.2.2. JSON形式の場合

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"dx3jXMk542T+tJCvIldpUQ=="
  },
  "printRequestIdList":{
    "printRequestId":[
      "2064-1"
    ]
  }
}
```

レスポンス 正常終了

```
{
  "resultCode":"S0-0001",
  "printDataList":{
    "printDataDetail":[
      {
        "printRequestId":"2064-1",
        "createDate":"2022/08/0118:22:24",
        "resultCode":"S0-0001",
        "url":"https://dummy.sagawa-exp.co.jp/"
      }
    ]
  }
}
```

レスポンス エラー有（ファイル取得有効期限切れ）

```
{
  "resultCode":"E8-0001",
  "printDataList":{
    "printDataDetail":[
      {
        "printRequestId":"11-1",
        "createDate":"2022/07/2115:53:48",
        "resultCode":"E2-0013"
      }
    ]
  }
}
```

4.インターフェース定義

4.4.利用実績参照API I-F

リクエスト

モデル名	項番	階層	項目名		タグ名	属性	桁数	備考	必須	入力例
利用実績参照 リクエスト	1	1	利用実績参照リクエスト	-	riyouJissekiRequest			XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	ユーザー認証	-	customerAuth				◎	
	1-1-1	3	カスタマーID		customerId	半角文字	50		◎	12345678
	1-1-2	3	ログインパスワード		loginPassword	半角文字	100		◎	dx3jXMk542T+tJCvIldpUQ==
	1-2	2	期間FROM		startDate	半角文字	20	yyyyMMdd	◎	20220701
	1-3	2	期間TO		endDate	半角文字	20	yyyyMMdd	◎	20220731

レスポンス

モデル名	項番	階層	項目名		タグ名	属性	桁数	備考	必須	出力例
利用実績参照 レスポンス	1	1	利用実績参照レスポンス	-	riyouJissekiResponse			XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	カスタマーID		customerId	半角文字	50	リクエスト時の値を返却	◎	12345678
	1-2	2	期間FROM		startDate	半角文字	20	リクエスト時の値を返却	◎	20220701
	1-3	2	期間TO		endDate	半角文字	20	リクエスト時の値を返却	◎	20220731
	1-4	2	件数		count	半角数字	8	リクエスト期間中の総利用実績件数	◎	1984
	1-5	2	処理結果コード		resultCode	半角文字	8	リクエストの全体の処理結果を返却	◎	S0-0001
	1-6	2	出荷実績リスト	-	shippingDataList					
	1-6-1	3	出荷実績明細	-	shippingDataDetail			リクエスト期間中の利用実績件数(問合番号発行数)を返却します		
	1-6-1-1	4	問合番号発行数		outputShippingCount	半角数字	8	リクエスト期間中の問合番号発行数		1984
	1-6-1-2	4	配送会社コード		deliveryCode	半角文字	4	佐川急便のコードを返却 詳細は5.1.パラメータコード定義参照		0001

4.インターフェース定義

4.4.利用実績参照API I-F 例

4.3.1. XML形式の場合

リクエスト

```
<?xml version="1.0" encoding="UTF-8"?>
<riyouJissekiRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword><![CDATA[dx3jXMk542T+tjCvIldpUQ==]]></loginPassword>
  </customerAuth>
  <startDate>20220101</startDate>
  <endDate>20221231</endDate>
</riyouJissekiRequest>
```

リクエスト

```
<?xml version="1.0" encoding="UTF-8"?>
<riyouJissekiRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword><![CDATA[dx3jXMk542T+tjCvIldpUQ==]]></loginPassword>
  </customerAuth>
  <startDate></startDate>
  <endDate>20221231</endDate>
</riyouJissekiRequest>
```

レスポンス 正常終了

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<riyouJissekiResponse>
  <customerId>12345678</customerId>
  <startDate>20220101</startDate>
  <endDate>20221231</endDate>
  <count>1984</count>
  <resultCode>S0-0001</resultCode>
  <shippingDataList>
    <shippingDataDetail>
      <outputShippingCount>1984</outputShippingCount>
      <deliveryCode>0001</deliveryCode>
    </shippingDataDetail>
  </shippingDataList>
</riyouJissekiResponse>
```

レスポンス エラー有

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<riyouJissekiResponse>
  <customerId>12345678</customerId>
  <startDate></startDate>
  <endDate>20221231</endDate>
  <count>0</count>
  <resultCode>E1-0001</resultCode>
</riyouJissekiResponse>
```

4.インターフェース定義

4.4.利用実績参照API I-F 例

4.3.2. JSON形式の場合

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"dx3jXMk542T+tJCvIldpUQ=="
  },
  "startDate":"20220101",
  "endDate":"20221231"
}
```

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"dx3jXMk542T+tJCvIldpUQ=="
  },
  "startDate": "",
  "endDate":"20221231"
}
```

レスポンス 正常終了

```
{
  "customerId":"12345678",
  "startDate":"20220101",
  "endDate":"20221231",
  "count":"1984",
  "resultCode":"S0-0001",
  "shippingDataList":{
    "shippingDataDetail":[
      {
        "outputShippingCount":"1984",
        "deliveryCode":"0001"
      }
    ]
  }
}
```

レスポンス エラー有

```
{
  "customerId":"12345678",
  "startDate": "",
  "endDate":"20221231",
  "count":"0",
  "resultCode":"E1-0001"
}
```

4.インターフェース定義

4.5.送り状再発行依頼API I-F

リクエスト

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	入力例
送り状再発行依頼 リクエスト	1	1	送り状再発行リクエスト	-	retryPrintRequest		XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	ユーザー認証	-	customerAuth			◎	
	1-1-1	3	カスタマーID		customerId	半角文字	50	◎	
	1-1-2	3	ログインパスワード		loginPassword	半角文字	100	◎	
	1-2	2	下敷画像表示フラグ		backLayerFlg	半角数字	1		帳票の背景画像をPDFに表示させるかのフラグ 0/null/空白 - 非表示 1 - 表示
	1-3	2	問合番号リスト	-	shippingNumberList			◎	送り状再発行対象のデータが複数ある場合は繰り返し指定可能
	1-3-1	3	問合番号		shippingNumber	半角文字	50	◎	

レスポンス

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	出力例
送り状再発行依頼 レスポンス	1	1	送り状再発行レスポンス	-	retryPrintResponse		XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	処理結果コード		resultCode	半角文字	8	◎	リクエスト全体の処理結果
	1-2	2	発行受付ID		printRequestId	半角文字	255		リクエスト全体の処理結果が正常の場合のみ返却 ※リクエストの問合番号リストに1つでもエラーが存在する場合は発行されません 送り状PDF再発行の受付番号
	1-3	2	エラー問合番号リスト	-	errorShippingNumberList				
	1-3-1	3	エラー問合番号明細	-	errorShippingNumberDetail				処理結果コードが正常の場合は返却されません 問合番号が複数ある場合は繰り返し返却
	1-3-1-1	4	問合番号		shippingNumber	半角文字	50		
	1-3-1-2	4	結果コードリスト	-	resultCodeList				
	1-3-1-2-1	5	処理結果コード		resultCode	半角文字	8		処理結果コードが複数ある場合は、繰り返し返却

4.インターフェース定義

4.5.送り状再発行依頼API I-F例

4.4.1. XML形式の場合

リクエスト

```
<?xml version="1.0" encoding="UTF-8"?>
<retryPrintRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword>dx3jXMk542T+tJCvIldpUQ==</loginPassword>
  </customerAuth>
  <backLayerFlg>1</backLayerFlg>
  <shippingNumberList>
    <shippingNumber>7777777777</shippingNumber>
    <shippingNumber>8888888888</shippingNumber>
    <shippingNumber>9999999999</shippingNumber>
  </shippingNumberList>
</retryPrintRequest>
```

レスポンス 正常終了

```
<?xml version="1.0" encoding="UTF-8"?>
<retryPrintResponse>
  <resultCode>S0-0001</resultCode>
  <printRequestId>2084-1</printRequestId>
</retryPrintResponse>
```

レスポンス エラー有

```
<?xml version="1.0" encoding="UTF-8"?>
<retryPrintResponse>
  <resultCode>E8-0001</resultCode>
  <errorShippingNumberList>
    <errorShippingNumberDetail>
      <shippingNumber>9999999999</shippingNumber>
      <resultCodeList>
        <resultCode>E1-0030</resultCode>
      </resultCodeList>
    </errorShippingNumberDetail>
  </errorShippingNumberList>
</retryPrintResponse>
```

4.インターフェース定義

4.5.送り状再発行依頼API I-F例

4.4.2. JSON形式の場合

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"dx3jXMk542T+tjCvIldpUQ=="
  },
  "backLayerFlg":"1",
  "shippingNumberList":{
    "shippingNumber":[
      "777777777777",
      "888888888888",
      "999999999999"
    ]
  }
}
```

レスポンス 正常終了

```
{
  "resultCode":"S0-0001",
  "printRequestId":"2084-1"
}
```

レスポンス エラー有

```
{
  "resultCode":"E8-0001",
  "errorShippingNumberList":{
    "errorShippingNumberDetail":[
      {
        "shippingNumber":"999999999999",
        "resultCodeList":{
          "resultCode":[
            "E1-0030"
          ]
        }
      }
    ]
  }
}
```

4.インターフェース定義

4.6.佐川急便マスタ参照API I-F

リクエスト

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	入力例
佐川急便 マスタ参照 リクエスト	1	1	佐川急便マスタ参照リクエスト	-	checkAddressRequest		XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	ユーザー認証	-	customerAuth			◎	
	1-1-1	3	カスタマーID	customerId	半角文字	50		◎	12345678
	1-1-2	3	ログインパスワード	loginPassword	半角文字	100		◎	dx3jXMk542T+tCvIdpUQ==
	1-2	2	リクエスト郵便番号	requestYubin	半角数字	7	ハイフン無し	○	
	1-3	2	リクエスト住所	requestAddress	指定なし	75		○	
	1-4	2	配送会社コード	deliveryCode	半角文字	4	下記「佐川急便」のコードを設定 0001/null/空白 - 佐川急便		0001

レスポンス

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	出力例
佐川急便 マスタ参照 レスポンス	1	1	佐川急便マスタ参照レスポンス	-	checkAddressResponse		XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	処理結果コード	resultCode	半角文字	8	リクエスト全体の処理結果	◎	
	1-2	2	リクエスト郵便番号	requestYubin	半角数字	7	リクエスト時の値を返却		
	1-3	2	リクエスト住所	requestAddress	指定なし	75	リクエスト時の値を返却		
	1-4	2	配送会社コード	deliveryCode	半角文字	4	リクエスト時の値を返却		
	1-5	2	住所リスト	-	addressList		処理結果コードが異常の場合は、返却されない		
	1-5-1	3	住所情報	-	addressInfo		リクエスト郵便番号に該当する住所情報を返却 複数ある場合は繰り返し返却		
	1-5-1-1	4	都道府県	to道府県Name	指定なし	16			
	1-5-1-2	4	市区郡町村名称	shikuchosonName	指定なし	40			
	1-5-1-3	4	町域名称	choikiName	指定なし	40			
	1-6	2	郵便番号リスト	-	yubinList		処理結果コードが異常の場合は、返却されない リクエスト住所に該当する郵便番号を返却 複数ある場合は繰り返し返却		
	1-6-1	3	郵便番号	yubin	半角数字	7	ハイフンなし		
	1-7	2	営業所コード	depotCode	半角文字	10	リクエスト住所の担当となる営業所コード 営業所情報は住所が一致している場合のみ返却		
	1-8	2	営業所名	depotName	指定なし	60	住所情報にヒモづく担当営業所名称 営業所情報は住所が一致している場合のみ返却		
	1-9	2	営業所電話番号	depotTel	半角文字	20	住所情報にヒモづく担当営業所電話番号 営業所情報は住所が一致している場合のみ返却		

4.インターフェース定義

4.6.佐川急便マスタ参照API I-F例

4.5.1. XML形式の場合

リクエスト

```
<?xml version="1.0" encoding="UTF-8"?>
<checkAddressRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword><![CDATA[dx3jXMk542T+UCvIldpUQ=]]></loginPassword>
  </customerAuth>
  <requestYubin>4520961</requestYubin>
  <requestAddress>愛知県清須市春日四番割</requestAddress>
</checkAddressRequest>
```

レスポンス

```
<?xml version="1.0" encoding="UTF-8"?>
<checkAddressRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword><![CDATA[dx3jXMk542T+UCvIldpUQ=]]></loginPassword>
  </customerAuth>
  <requestYubin>9160143</requestYubin>
  <requestAddress></requestAddress>
</checkAddressRequest>
```

レスポンス 郵便番号と住所が一致

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<checkAddressResponse>
  <resultCode>S0-0001</resultCode>
  <requestYubin>4520961</requestYubin>
  <requestAddress>愛知県清須市春日四番割</requestAddress>
  <addressList>
    <addressInfo>
      <todofukenName>愛知県</todofukenName>
      <shikuchosonName>清須市</shikuchosonName>
      <choikiName>春日四番割</choikiName>
    </addressInfo>
  </addressList>
  <yubinList>
    <yubin>4520961</yubin>
  </yubinList>
  <depotCode>6012</depotCode>
  <depotName>一宮</depotName>
  <depotTel>0120-700-850</depotTel>
</checkAddressResponse>
```

レスポンス 郵便番号のみでのリクエスト結果

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<checkAddressResponse>
  <resultCode>S0-0001</resultCode>
  <requestYubin>9160143</requestYubin>
  <requestAddress></requestAddress>
  <addressList>
    <addressInfo>
      <todofukenName>福井県</todofukenName>
      <shikuchosonName>丹生郡越前町</shikuchosonName>
      <choikiName>漆本</choikiName>
    </addressInfo>
    <addressInfo>
      <todofukenName>福井県</todofukenName>
      <shikuchosonName>丹生郡越前町</shikuchosonName>
      <choikiName>宇田</choikiName>
    </addressInfo>
  </addressList>
  <depotCode>5020</depotCode>
  <depotName>福井</depotName>
  <depotTel>0120-700-850</depotTel>
</checkAddressResponse>
```

4.インターフェース定義

4.6.佐川急便マスタ参照API I-F例

リクエスト

```
<?xml version="1.0" encoding="UTF-8"?>
<checkAddressRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword><![CDATA[dx3jXMk542T+UCvIldpUQ=]]></loginPassword>
  </customerAuth>
  <requestYubin>4110000</requestYubin>
  <requestAddress></requestAddress>
</checkAddressRequest>
```

リクエスト

```
<?xml version="1.0" encoding="UTF-8"?>
<checkAddressRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword><![CDATA[dx3jXMk542T+UCvIldpUQ=]]></loginPassword>
  </customerAuth>
  <requestYubin>6048102</requestYubin>
  <requestAddress>東京都江東区新砂</requestAddress>
</checkAddressRequest>
```

レスポンス 郵便番号のみリクエスト結果(店候補が複数あり、取得できなかった場合はワーニングとなります)

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<checkAddressResponse>
  <resultCode>W0-0002</resultCode>
  <requestYubin>4110000</requestYubin>
  <requestAddress></requestAddress>
  <addressList>
    <addressInfo>
      <todofukenName>静岡県</todofukenName>
      <shikuchosonName>駿東郡清水町</shikuchosonName>
      <choikiName></choikiName>
    </addressInfo>
    <addressInfo>
      <todofukenName>静岡県</todofukenName>
      <shikuchosonName>三島市</shikuchosonName>
      <choikiName></choikiName>
    </addressInfo>
    <addressInfo>
      <todofukenName>静岡県</todofukenName>
      <shikuchosonName>駿東郡長泉町</shikuchosonName>
      <choikiName></choikiName>
    </addressInfo>
  </addressList>
  <depotCode></depotCode>
  <depotName></depotName>
  <depotTel></depotTel>
</checkAddressResponse>
```

レスポンス 郵便番号と住所不一致

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<checkAddressResponse>
  <resultCode>E2-0001</resultCode>
  <requestYubin>6048102</requestYubin>
  <requestAddress>東京都江東区新砂</requestAddress>
</checkAddressResponse>
```

4.インターフェース定義

4.6.佐川急便マスタ参照API I-F例

4.5.2. JSON形式の場合

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"dx3jXMk542T+tjCvIldpUQ=="
  },
  "requestYubin":"4520961",
  "requestAddress":"愛知県清須市春日四番割"
}
```

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"dx3jXMk542T+tjCvIldpUQ=="
  },
  "requestYubin":"9160143",
  "requestAddress":""
}
```

レスポンス 郵便番号と住所が一致

```
{
  "resultCode":"S0-0001",
  "requestYubin":"4520961",
  "requestAddress":"愛知県清須市春日四番割",
  "addressList":{
    "addressInfo":[
      {
        "todofukenName":"愛知県",
        "shikuchosonName":"清須市",
        "choikiName":"春日四番割"
      }
    ]
  },
  "yubinList":{
    "yubin":[
      "4520961"
    ]
  },
  "depotCode":"6012",
  "depotName":"一宮",
  "depotTel":"0120-700-850"
}
```

レスポンス 郵便番号のみでのリクエスト結果

```
{
  "resultCode":"S0-0001",
  "requestYubin":"9160143",
  "requestAddress":"","
  "addressList":{
    "addressInfo":[
      {
        "todofukenName":"福井県",
        "shikuchosonName":"丹生郡越前町",
        "choikiName":"漆本"
      },
      {
        "todofukenName":"福井県",
        "shikuchosonName":"丹生郡越前町",
        "choikiName":"宇田"
      }
    ]
  },
  "depotCode":"5020",
  "depotName":"福井",
  "depotTel":"0120-700-850"
}
```

4.インターフェース定義

4.6.佐川急便マスタ参照API I-F例

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"dx3jXMk542T+tjCvIldpUQ=="
  },
  "requestYubin":"4110000",
  "requestAddress":""
}
```

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"dx3jXMk542T+tjCvIldpUQ=="
  },
  "requestYubin":"6048102",
  "requestAddress":"東京都江東区新砂"
}
```

レスポンス 郵便番号のみリクエスト結果(店候補が複数あり、取得できなかった場合はワーニングとなります)

```
{
  "resultCode":"W0-0002",
  "requestYubin":"4110000",
  "requestAddress":"",
  "addressList":{
    "addressInfo":[
      {
        "todofukenName":"静岡県",
        "shikuchosonName":"駿東郡清水町",
        "choikiName":""
      },
      {
        "todofukenName":"静岡県",
        "shikuchosonName":"三島市",
        "choikiName":""
      },
      {
        "todofukenName":"静岡県",
        "shikuchosonName":"駿東郡長泉町",
        "choikiName":""
      }
    ]
  },
  "depotCode":"",
  "depotName":"",
  "depotTel":""
}
```

レスポンス 郵便番号と住所不一致

```
{
  "resultCode":"E2-0001",
  "requestYubin":"6048102",
  "requestAddress":"東京都江東区新砂"
}
```

4.インターフェース定義

4.7.荷物受渡書・出荷明細書発行API I-F

リクエスト

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	入力例
荷物受渡書・ 出荷明細書発行 リクエスト	1	1	荷物受渡書・出荷明細書発行リクエスト	- ukewatashiMeisaiRequest			XML形式の場合必須、JSON形式の場合不要	○	
	1-1	2	ユーザー認証	- customerAuth				◎	
	1-1-1	3	カスタマーID	customerId	半角文字	50		◎	12345678
	1-1-2	3	ログインパスワード	loginPassword	半角文字	100		◎	dx3jXMk542T+tjCvIldpUQ==
	1-2	2	配送会社コード	deliveryCode	半角文字	4	下記「佐川急便」のコードを設定 0001/null/空白 - 佐川急便		0001
	1-3	2	帳票コード	chohyoCode	半角文字	4	荷物受渡書の帳票コードを設定	◎	U402
	1-4	2	下敷画像表示フラグ	backLayerFlg	半角数字	1	帳票の背景画像をPDFに表示させるかのフラグ 0/null/空白 - 非表示 1 - 表示		1
	1-5	2	荷物受渡書・出荷明細書出力タイプ	ukewatashiMeisaiType	半角数字	3	出力する荷物受渡書と出荷明細書の組合タイプを設定 5.1/パラメータコード定義書参照	◎	
	1-6	2	荷物受渡書・出荷明細書情報リスト	- ukewatashiMeisaiList					
	1-6-1	3	荷物受渡書・出荷明細書情報	- ukewatashiMeisaiDetail					
	1-6-1-1	4	顧客コード	kokyakuCode	半角数字	12		◎	
	1-6-1-2	4	出荷日	shukkaDate	半角数字	8			
	1-6-1-3	4	総出荷個数	totalKosu	半角数字	8			
	1-6-1-4	4	総出荷件数	totalKensu	半角数字	8			
	1-6-1-5	4	総削除個数	totalSakujoKosu	半角数字	8	使用不可		
	1-6-1-6	4	総削除件数	totalSakujoKensu	半角数字	8	使用不可		
	1-6-1-7	4	問合せ番号リスト	- shippingNumberList					
	1-6-1-7-1	5	問合せ番号	shippingNumber	半角文字	50			

4.インターフェース定義

4.7.荷物受渡書・出荷明細書発行API I-F

レスポンス

モデル名	項番	階層	項目名	タグ名	属性	桁数	備考	必須	出力例
荷物受渡書・ 出荷明細書発行 レスポンス	1	1	荷物受渡書・出荷明細書発行レスポンス -	ukewatashiMesaiResponse			XML形式の場合必須、JSON形式の場合不要	○	
	1-2	2	帳票コード	chohyoCode	半角文字	4	荷物受渡書の帳票コードを設定	◎	U402
	1-3	2	荷物受渡書・出荷明細書出力タイプ	ukewatashiMeisaiType	半角数字	3	出力する荷物受渡書と出荷明細書の組合タイプを設定 5.1.パラメータコード定義書参照 空白の場合は自動で、荷物受渡書のみ出力する	◎	
	1-4	2	処理結果コード	resultCode	半角文字	8	処理結果コード	◎	S0-0001
	1-5	2	発行受付ID	printRequestId	半角文字	255	荷物受渡書のPDF発行の受付番号		
	1-6	2	荷物受渡書・出荷明細書情報リスト -	ukewatashiMeisaiList					
	1-6-1	3	荷物受渡書・出荷明細書情報 -	ukewatashiMeisaiDetail					
	1-6-2	4	顧客コード	kokyakuCode	半角数字	12	出荷明細書に記載されている顧客コード		
	1-6-3	4	問合番号記載数	describeShippingCount	半角数字	8	出荷明細書に記載されている問合番号の合計件数		999
	1-6-4	4	結果コードリスト -	resultCodeList					
	1-6-4-1	5	処理結果コード	resultCode	半角文字	8	処理結果コードが複数ある場合繰り返し返却		

4.インターフェース定義

4.7.荷物受渡書・出荷明細書発行API I-F例

4.6.1. XML形式の場合

リクエスト

```
<?xml version="1.0" encoding="UTF-8"?>
<ukewatashiMeisaiRequest>
  <customerAuth>
    <customerId>12345678</customerId>
    <loginPassword>
      <![CDATA[dx3jXmk542T+tjCvIldpUQ==]]>
    </loginPassword>
  </customerAuth>
  <deliveryCode>0001</deliveryCode>
  <chohyoCode>U402</chohyoCode>
  <backLayerFlg>0</backLayerFlg>
  <ukewatashiMeisaiType>001</ukewatashiMeisaiType>
  <ukewatashiMeisaiList>
    <ukewatashiMeisaiDetail>
      <kokyakuCode>999999999999</kokyakuCode>
      <shukkaDate>20220801</shukkaDate>
      <totalKosu>99999999</totalKosu>
      <totalKensu>99999999</totalKensu>
      <shippingNumberList>
        <shippingNumber>999999999999</shippingNumber>
      </shippingNumberList>
    </ukewatashiMeisaiDetail>
  </ukewatashiMeisaiList>
</ukewatashiMeisaiRequest>
```

レスポンス

正常終了

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<ukewatashiMeisaiResponse>
  <chohyoCode>U402</chohyoCode>
  <ukewatashiMeisaiType>000</ukewatashiMeisaiType>
  <resultCode>S0-0001</resultCode>
  <printRequestId>2064-1</printRequestId>
  <ukewatashiMeisaiList>
    <ukewatashiMeisaiDetail>
      <kokyakuCode>999999999999</kokyakuCode>
      <describeShippingCount>1</describeShippingCount>
      <resultCodeList>
        <resultCode>S0-0001</resultCode>
      </resultCodeList>
    </ukewatashiMeisaiDetail>
  </ukewatashiMeisaiList>
</ukewatashiMeisaiResponse>
```

レスポンス

エラーあり

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<ukewatashiMeisaiResponse>
  <chohyoCode>U402</chohyoCode>
  <ukewatashiMeisaiType>000</ukewatashiMeisaiType>
  <resultCode>E8-0001</resultCode>
  <printRequestId></printRequestId>
  <ukewatashiMeisaiList>
    <ukewatashiMeisaiDetail>
      <kokyakuCode>999999999999</kokyakuCode>
      <describeShippingCount></describeShippingCount>
      <resultCodeList>
        <resultCode>E1-0084</resultCode>
      </resultCodeList>
    </ukewatashiMeisaiDetail>
  </ukewatashiMeisaiList>
</ukewatashiMeisaiResponse>
```

4.インターフェース定義

4.7.荷物受渡書・出荷明細書発行API I-F例

4.6.2. JSON形式の場合

リクエスト

```
{
  "customerAuth":{
    "customerId":"12345678",
    "loginPassword":"¥ndx3jXMk542T+UCvIldpUQ==¥n"
  },
  "deliveryCode":"0001",
  "chohyoCode":"U402",
  "backLayerFlg":"0",
  "ukewatashiMeisaiType":"001",
  "ukewatashiMeisaiList":{
    "ukewatashiMeisaiDetail":[
      {
        "kokyakuCode":"999999999999",
        "shukkaDate":"20220801",
        "totalKosu":"99999999",
        "totalKensu":"99999999",
        "shippingNumberList":{
          "shippingNumber":[
            "999999999999"
          ]
        }
      }
    ]
  }
}
```

レスポンス

正常終了

```
{
  "chohyoCode":"U402",
  "ukewatashiMeisaiType":"000",
  "resultCode":"S0-0001",
  "printRequestId":"2064-1",
  "ukewatashiMeisaiList":{
    "ukewatashiMeisaiDetail":[
      {
        "kokyakuCode":"999999999999",
        "describeShippingCount":"1",
        "resultCodeList":{
          "resultCode":[
            "S0-0001"
          ]
        }
      }
    ]
  }
}
```

レスポンス

エラーあり

```
{
  "chohyoCode":"U402",
  "ukewatashiMeisaiType":"000",
  "resultCode":"E8-0001",
  "printRequestId":"",
  "ukewatashiMeisaiList":{
    "ukewatashiMeisaiDetail":[
      {
        "kokyakuCode":"999999999999",
        "describeShippingCount":"",
        "resultCodeList":{
          "resultCode":[
            "E1-0084"
          ]
        }
      }
    ]
  }
}
```

5.コード定義

5.1.パラメータコード定義

項目名	タグ名	値	内容	備考
便種コード	binsyuCode	000	陸便	
		030	航空便	
		140	クール冷蔵	
		141	クール冷蔵(航空便)	
		150	クール冷凍	
		151	クール冷凍(航空便)	
代引支払方法区分	daibikiType	null	なし	
		0	なんでも決済	届先への代引支払方法をカード、現金どちらでも対応
		1	現金	届先への代引支払方法を現金払いのみ対応
		2	クレジットカード・デビットカード	届先への代引支払方法をクレジットカード・デビットカード払いのみ対応
重量1	weight1	60	2Kg(サイズ60)	サイズ指定がない場合はコード60とコード80が選択されます
重量2	weight2	80	5Kg(サイズ80)	
		100	10Kg(サイズ100)	
		140	20Kg(サイズ140)	
		160	30Kg(サイズ160)	
配達時間指定コード	shiteiTimeCode	空白/null	時間帯指定なし	
		00	時間帯指定なし	
		01	午前中	
		12	12:00～14:00	
		14	14:00～16:00	
		16	16:00～18:00	
		18	18:00～20:00	
		19	19:00～21:00	
		04	18:00～21:00	
シール1	careSeal1	011	取扱注意	リクエスト時に選択可能なシールです その他ケアマークシールはシステム側でリクエスト内容から判断し自動的に付与します
シール2	careSeal2	013	天地無用	
シール3	careSeal3	012	貴重品	
配送会社コード	deliveryCode	0001/null/空白	佐川急便	
出力レベル	outputLevel	000	エラー＆ワーニング精査	エラー精査、ワーニング精査を行います
		900	エラー精査	エラー精査のみ行い、ワーニング精査は行いません
送り状コード	okuriCode	A501	佐川急便A5サイズ圧着式送り状	
		A501C06	佐川急便A5サイズ圧着式送り状(透かし文字無版)	
		L02C04	佐川急便統一圧着サーマル送り状	
		L02C05	佐川急便統一圧着サーマル送り状(透かし文字無版)	
		L01C07	佐川急便ケアマーク入圧着サーマル送り状（4種）	
		L01C08	佐川急便ケアマーク入圧着サーマル送り状（4種）(透かし文字無版)	
		U402	荷物受渡書・出荷明細書	
荷物受渡書・出荷明細書出力タイプ	ukewatashiMeisaiType	000	荷物受渡書	
		001	荷物受渡書・出荷明細書	
		002	出荷明細書	
元着コード	motoChakuCode	0/null/空白	元払い	着払いのご利用いただけません

5.コード定義

5.2.処理結果コード

コード形式[XY-9999]

X	S	正常終了	正常終了
	E	エラー	エラー
	W	ワーニング	ワーニング

Y	0	正常終了
	1	リクエスト内容精査結果
	2	判定内容結果
	3	認証内容結果
	8	処理内容結果
	9	システム結果

利用API一覧

No.	API名称
1	送り状発行API
2	即時発行API
3	ファイル存在確認API
4	送り状再発行依頼API
5	利用実績API
6	佐川急便マスタ参照API
7	荷物受渡書・出荷明細書発行API

返却処理結果コード

◎	全APIにて共通で処理結果コードが返却されます
○	ご利用のAPIごとに異なる処理結果コードが返却されます

No.	処理結果コード	処理結果内容	備考	利用API No.						
				1	2	3	4	5	6	7
0	x0-xxxx	正常終了								
0-1	S0-0001	正常終了	正常に処理が完了した場合、表示されます	◎	◎	◎	◎	◎	◎	◎
0-2	W0-0002	正常終了	正常に処理が完了したがワーニング情報がある場合、表示されます						○	
1	x1-xxxx	リクエスト内容精査結果								
1-1	E1-0001	必須項目に値がありません	必須項目に値がなかった場合に返却されます	◎	◎	◎	◎	◎		◎
1-2	E1-0002	管理番号が重複しています	管理番号が重複していた場合返却されます	○	○					
1-3	E1-0003	未登録の顧客コードです	システムに登録されていない顧客コードの場合返却されます	○	○					
1-4	E1-0004	個口数が正しくありません	配送個口数の値が正しくない場合返却されます	○	○					
1-5	E1-0005	便種コードが正しくありません	便種コードの値が正しくない場合返却されます	○	○					
1-6	E1-0006	配達時間指定が正しくないコードです	配達時間指定の値が正しくない場合返却されます	○	○					
1-7	E1-0007	対象の顧客コードが代引契約をしていません	対象の顧客コードが代引契約をしていない場合返却されます	○	○					
1-8	E1-0008	配達指定日の日付が正しくありません	配達指定日の値が正しくない場合返却されます	○	○					
1-9	E1-0009	発送日の日付が正しくありません	発送日の値が正しくない場合返却されます	○	○					
1-10	E1-0010	シール1のコードが正しくありません	ケアマークシール1の値が正しくない場合返却されます	○	○					
1-11	E1-0011	シール2のコードが正しくありません	ケアマークシール2の値が正しくない場合返却されます	○	○					
1-12	E1-0012	シール3のコードが正しくありません	ケアマークシール3の値が正しくない場合返却されます	○	○					
1-13	E1-0013	代金引換が正しくありません	代引金額の値が正しくない場合返却されます	○	○					
1-14	E1-0014	対象の顧客コードは運用保険を利用できません	対象の顧客コードが運用保険を利用していない場合返却されます	○	○					
1-15	E1-0015	依頼主住所1～3で文字数をオーバーしています	依頼主住所1～3で文字数をオーバーしていた場合返却されます	○	○					
1-16	E1-0016	依頼主氏名1～2で文字数をオーバーしています	依頼主氏名1～2で文字数をオーバーしていた場合返却されます	○	○					
1-17	E1-0017	依頼主住所1に値がありません	依頼主住所1に値がない場合返却されます	○	○					
1-18	E1-0018	依頼主氏名1に値がありません	依頼主氏名1に値がない場合返却されます	○	○					
1-19	E1-0019	お届け先住所1に値がありません	届先住所1に値がない場合返却されます	○	○					
1-20	E1-0020	お届け先氏名1に値がありません	届先氏名1に値がない場合返却されます	○	○					
1-21	E1-0021	記事1～6の文字数がオーバーしています	記事1～6で文字数をオーバーしていた場合返却されます	○	○					
1-22	E1-0022	フラグの値が正しくありません	フラグ関連の項目にて指定外の値の場合返却されます	○	○					○
1-23	E1-0023	重量1または重量2のコードが正しくありません	重量1、重量2の値が正しくない場合返却されます	○	○					
1-24	E1-0024	発行受付IDが重複しています	発行受付IDを重複リクエストした場合返却されます			○				
1-25	E1-0025	該当する発行受付IDがありません	発行受付IDが該当しない場合返却されます			○				

5.コード定義

5.2.処理結果コード

1-26	E1-0026	該当する問合番号がありません	問合番号が該当しない場合返却されます					○				
1-27	E1-0027	元着コードが正しくありません	元着コードの値が正しくない場合返却されます	○	○							
1-28	E1-0028	配送料金が着払の場合は代引が利用できません。	配送料金が着払いと代引の併用をしていた場合返却されます	○	○							
1-29	E1-0029	配達指定日が過去日付です	配達指定日が発送日翌日より過去日付だった場合返却されます	○	○							
1-30	E1-0030	再発行期間対象外です	再発行可能期間切れの問合番号(出荷データ)の場合返却されます					○				
1-31	E1-0032	代引支払方法区分が正しくありません	代引支払方法区分の値が正しくない場合返却されます	○	○							
1-32	E1-0035	文字数が正しくありません	文字数がオーバーしていた場合返却されます	⓪	⓪	⓪	⓪	⓪	⓪	⓪	⓪	⓪
1-33	E1-0036	同じシールは2つ指定できません	ケアマークシール1〜3にて同じ値を指定した場合返却されます	○	○							
1-34	E1-0037	配達指定日が30日を超える未来日付です	配達指定日が発送日(発送日に入力がない場合送り状発行日付)より30日を超える場合返却されます	○	○							
1-35	E1-0038	届先郵便番号が正しい値ではありません	届先郵便番号はハイフン無しで半角数字7桁ではない場合返却されます	○	○							
1-36	E1-0039	依頼主郵便番号が正しい値ではありません	依頼主郵便番号はハイフン無しで半角数字7桁ではない場合返却されます	○	○							
1-37	E1-0040	届先電話番号が正しい値ではありません	届先電話番号は数字またはハイフンではない場合返却されます	○	○							
1-38	E1-0041	依頼主電話番号が正しい値ではありません	依頼主電話番号は数字またはハイフンでない場合返却されます	○	○							
1-39	E1-0042	お届け先住所1〜3で文字数をオーバーしています	お届け先住所1〜3で文字数をオーバーしていた場合返却されます	○	○							
1-40	E1-0043	お届け先氏名1〜2で文字数をオーバーしています	お届け先氏名1〜2で文字数をオーバーしていた場合返却されます	○	○							
1-41	E1-0044	営業所止めでの時間帯指定は不可となります	営業所止めと時間帯指定の併用をしていた場合返却されます	○	○							
1-42	E1-0045	営業所止めでの配達指定は不可となります	営業所止めと配達指定の併用をしていた場合返却されます	○	○							
1-43	E1-0046	発送日が30日を超える未来日付です	発送日が今日日付より30日を超える場合返却されます	○	○							
1-44	E1-0047	送り状コードが正しくありません	送り状コードの値が正しくない場合返却されます	○	○							
1-45	E1-0048	出力レベルが正しくありません	出力レベルの値が正しくない場合返却されます	○	○							
1-46	E1-0049	管理番号が正しくありません	管理番号の値が半角文字ではない、または文字数がオーバーしていた場合返却されます	○	○							
1-47	E1-0050	顧客コードが正しくありません	顧客コードの値が半角文字ではない、または文字数がオーバーしていた場合返却されます	○	○							○
1-48	E1-0051	依頼主メールアドレスが正しくありません	依頼主メールアドレスが半角文字ではない、または文字数がオーバーしていた場合返却されます	○	○							
1-49	E1-0052	お届けメールアドレスが正しくありません	お届けメールアドレスが半角文字ではない、または文字数がオーバーしていた場合返却されます	○	○							
1-50	E1-0053	代引消費税が正しくありません	代引消費税の値が正しくない場合返却されます	○	○							
1-51	E1-0054	保険金額が正しくありません	保険金額の値が正しくない場合返却されます	○	○							
1-52	E1-0055	営業所コードが正しくありません	営業所コードが正しくない場合返却されます	○	○							
1-53	E1-0057	代引支払方法区分は対応できません	リクエストの代引支払方法区分が未登録の場合返却されます	○	○							
1-54	E1-0058	発送日が過去日付です	発送日が送り状発行した日付今日日付より過去日付だった場合返却されます	○	○							
1-55	E1-0059	再印刷する問合番号が重複しています	問合番号を重複リクエストした場合返却されます					○				
1-56	E1-0060	指定期間が正しくありません	指定期間が正しくない場合返却されます						○			
1-57	E1-0061	郵便番号または住所に値がありません	郵便番号または住所に値がない場合返却されます								○	
1-58	E1-0062	数値が正しくありません	文字または限度数をオーバーした値をリクエストした場合返却されます									○
1-59	E1-0063	出荷日が正しくありません	出荷日の値が正しくない場合返却されます									○
1-60	E1-0064	配送会社コードが正しくありません	配送会社コードが正しくない場合返却されます									○
1-61	E1-0084	帳票コードが正しくありません	帳票コードが正しくない場合返却されます									○
1-62	E1-0085	荷物受渡書・出荷明細出力タイプが正しくありません	荷物受渡書・出荷明細出力タイプが正しくない場合返却されます									○
1-63	E1-0095	対象の送り状コードでの着払いの利用はできません	対象の送り状コードが着払いに対応していない場合返却されます	○	○							
2	x2-xxxx	判定内容結果										
2-1	E2-0001	郵便番号と住所の値の整合性がない場合返却されます	郵便番号と住所の値の整合性がない場合返却されます								○	
2-2	E2-0002	郵便番号から該当する住所はありません	リクエストの郵便番号から該当の値を取得できなかった場合返却されます	○	○						○	

5.コード定義

5.2.処理結果コード

2-3	E2-0003	住所から該当する郵便番号はありません	リクエストの住所から該当の値を取得できなかった場合返却されます							○	
2-4	E2-0004	お届け先の郵便番号と住所が一致していません	お届け先の郵便番号と住所が一致していない場合返却されます	○	○						
2-5	E2-0005	依頼主の郵便番号と住所が一致していません	依頼主の郵便番号と住所が一致していない場合返却されます	○	○						
2-6	E2-0006	対象の発行受付IDはまだ発行の準備ができていません	発行受付IDに該当するデータは作成中ですので、時間を置いて再度リクエストしてください			○					
2-7	E2-0007	代金引換不可エリアです	お届け先が代金引換サービス不可な地域の場合返却されます	○	○						
2-8	E2-0008	クール冷蔵不可エリアです	お届け先がクール冷蔵サービス不可な地域の場合返却されます	○	○						
2-9	E2-0009	クール冷凍不可エリアです	お届け先がクール冷凍サービス不可な地域の場合返却されます	○	○						
2-10	E2-0013	データ取得有効期間を過ぎています	データ取得有効期限が過ぎている場合返却されます			○					
2-11	E2-0014	リクエストできるレコード数をオーバーしています	リクエストできるレコード数をオーバーしています	○	○		○				○
2-12	W2-0001	時間帯指定不可エリアです	お届け先が時間帯指定サービス不可な地域の場合返却されます	○	○						
2-13	W2-0003	中継エリアです	お届け先が中継料金がかかる場合返却されます	○	○						
2-14	W2-0005	取得できなかった明細データがあります	取得できなかった明細データがある場合返却されます								○
3	x3-xxxx	認証内容結果									
3-1	E3-0001	アカウント認証が正しくありません	カスタマーIDとパスワードの認証に失敗した場合返却されます	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗
4	x8-xxxx	処理内容結果									
4-1	E8-0001	エラー有り	エラー内容を含む場合エラー有りとなります	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗
4-2	E8-0002	問合番号が取得できません	荷物問合番号を取得できなかった際に返却されます	○	○						
5	x9-xxxx	システム処理結果									
5-1	E9-0001	システムエラーです、管理者へお問い合わせください	弊社の問合せサポートまでお問合せお願い致します	⊗	⊗	⊗	⊗	⊗	⊗	⊗	⊗

